Red Hat OpenShift Serverless 1.35

Serverless Logic

Introduction to OpenShift Serverless Logic

Last Updated: 2025-09-10

Red Hat OpenShift Serverless 1.35 Serverless Logic

Introduction to OpenShift Serverless Logic

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

This document provides an overview of OpenShift Serverless Logic features.

Table of Contents

CHAPTER 1. GETTING STARTED 5
1.1. CREATING AND RUNNING WORKFLOWS WITH KNATIVE WORKFLOW PLUGIN 5
1.1.1. Creating a workflow 5
1.1.2. Running a workflow locally 5
1.2. DEPLOYMENT OPTIONS AND DEPLOYING WORKFLOWS 6
1.2.1. Deploying workflows using the Dev profile 6
1.2.2. Deploying workflows using the Preview profile 8
1.2.2.1. Configuring workflows in Preview profile 8
1.2.2.1.1. Configuring the workflow base builder image 8
1.2.2.1.2. Customization for the base builder Dockerfile 9
1.2.2.1.3. Changing resource requirements 11
1.2.2.1.4. Passing arguments to the internal builder 12
1.2.2.1.5. Setting environment variables in the internal builder 13
1.2.2.1.6. Changing the base builder image 14
1.2.2.2. Building and deploying your workflow 15
1.2.2.3. Verifying workflow deployment 16
1.2.2.4. Restarting a build 18
1.2.3. Deploying workflows using the GitOps profile 19
1.2.4. Editing a workflow 20
1.2.5. Testing a workflow 21
1.2.6. Troubleshooting a workflow 22
1.2.7. Deleting a workflow 22

CHAPTER 2. GLOBAL CONFIGURATION SETTINGS 23
2.1. PREREQUISITES 23
2.2. CUSTOMIZATION OF GLOBAL CONFIGURATIONS 23
2.2.1. Impact of global configuration changes 26
2.2.2. Customizing the base builder image 26

CHAPTER 3. MANAGING SERVICES 27
3.1. CONFIGURING OPENAPI SERVICES 27
3.1.1. OpenAPI function definition 27
3.1.2. Sending REST requests based on the OpenAPI specification 27
3.1.3. Configuring the endpoint URL of OpenAPI services 31
3.2. CONFIGURING OPENAPI SERVICES ENDPOINTS 32
3.2.1. Using URI alias 34
3.3. TROUBLESHOOTING SERVICES 36
3.3.1. Tracing HTTP requests and responses 36

CHAPTER 4. SUPPORTING SERVICES 38
4.1. JOB SERVICE 38
4.1.1. Job service leader election process 38
4.2. DATA INDEX SERVICE 38
4.2.1. GraphQL queries for workflow instances and jobs 39
4.2.1.1. Retrieve data from workflow instances 39
4.2.1.2. Retrieve data from jobs 39
4.2.1.3. Filter query results by using the where parameter 40
4.2.1.4. Sort query results by using the orderBy parameter 42
4.2.1.5. Limit the number of results by using the pagination parameter 43
4.3. MANAGING SUPPORTING SERVICES 43
4.3.1. Supporting services and workflow integration 43
4.3.2. Supporting services deployment with the SonataFlowPlatform CR 44

4.3.3. Supporting services scope 44
4.3.4. Supporting services persistence configurations 45
4.3.4.1. Ephemeral persistence configuration 45
4.3.4.2. PostgreSQL persistence configuration 45
4.3.4.3. Common PostgreSQL persistence configuration 46
4.3.5. Supporting services eventing system configurations 47
4.3.5.1. Platform-scoped eventing system configuration 47
4.3.5.2. Service-scoped eventing system configuration 48
4.3.5.3. Data Index eventing system configuration 48
4.3.5.4. Job Service eventing system configuration 49
4.3.5.5. Cluster-scoped eventing system configuration for supporting services 49
4.3.5.6. Eventing system configuration precedence rules for supporting services 50
4.3.5.7. Eventing system linking configuration 50
4.3.6. Advanced supporting services configurations 52
4.3.7. Cluster scoped supporting services 53

CHAPTER 5. MANAGING WORKFLOW PERSISTENCE **54**
5.1. CONFIGURING PERSISTENCE USING THE SONATAFLOWPLATFORM CR 54
5.2. CONFIGURING PERSISTENCE USING THE SONATAFLOW CR 56
5.3. PERSISTENCE CONFIGURATION PRECEDENCE RULES 57
5.4. PROFILE SPECIFIC PERSISTENCE REQUIREMENTS 57
5.5. DATABASE SCHEMA INITIALIZATION 58
5.5.1. Flyway configuration in the workflow ConfigMap 58
5.5.2. Flyway configuration using environment variables in the workflow container 58
5.5.3. Flyway configuration using SonataFlowPlatform properties 59
5.5.4. Initializing a manual database using DDL scripts 59

CHAPTER 6. WORKFLOW EVENTING SYSTEM **61**
6.1. PLATFORM-SCOPED EVENTING SYSTEM CONFIGURATION 61
6.2. WORKFLOW-SCOPED EVENTING SYSTEM CONFIGURATION 62
6.2.1. Outgoing eventing system configuration 62
6.2.2. Incoming eventing system configuration 63
6.3. CLUSTER-SCOPED EVENTING SYSTEM CONFIGURATION 64
6.4. EVENTING SYSTEM CONFIGURATION PRECEDENCE RULES 65
6.5. LINKING WORKFLOWS TO THE EVENTING SYSTEM 65

CHAPTER 7. CONFIGURING CUSTOM MAVEN MIRRORS **69**
7.1. ADDING A MAVEN MIRROR WHEN BUILDING WORKFLOWS 69
7.2. ADDING A MAVEN MIRROR WHEN DEPLOYING IN DEVELOPMENT MODE 70
7.3. CONFIGURING A MAVEN MIRROR ON A CUSTOM IMAGE 71

CHAPTER 8. MANAGING UPGRADES **73**
8.1. UPGRADING OPENSHIFT SERVERLESS LOGIC OPERATOR FROM VERSION 1.34.0 TO 1.35.0 73
8.1.1. Preparing for the upgrade 73
8.1.1.1. Deleting workflows with dev profile 73
8.1.1.2. Deleting and migrating workflows with the preview profile 74
8.1.1.3. Scaling down workflows with the gitops profile 74
8.1.1.4. Backing up the Data Index database 75
8.1.1.5. Backing up the Jobs Service database 75
8.1.2. Upgrading the OpenShift Serverless Logic Operator 75
8.1.3. Finalizing the upgrade 76
8.1.3.1. Finalizing the Data Index upgrade 76
8.1.3.2. Finalizing the Job Service upgrade 77
8.1.3.3. Redeploying workflows with the dev profile 77

8.1.3.4. Restoring workflows with the preview profile 77
8.1.3.5. Scaling up workflows with the gitops profile 78
8.1.3.6. Verifying the upgrade 78

CHAPTER 1. GETTING STARTED

1.1. CREATING AND RUNNING WORKFLOWS WITH KNATIVE WORKFLOW PLUGIN

You can create and run the OpenShift Serverless Logic workflows locally.

1.1.1. Creating a workflow

You can use the **create** command with **kn workflow** to set up a new OpenShift Serverless Logic project in your current directory.

Prerequisites

- You have installed the OpenShift Serverless Logic **kn-workflow** CLI plugin.

Procedure

1. Create a new OpenShift Serverless Logic workflow project by running the following command:

```
$ kn workflow create
```

By default, the generated project name is **new-project**. You can change the project name by using the **[-n|--name]** flag as follows:

Example command

```
$ kn workflow create --name my-project
```

1.1.2. Running a workflow locally

You can use the **run** command with **kn workflow** to build and run your OpenShift Serverless Logic workflow project in your current directory.

Prerequisites

- You have installed Podman on your local machine.
- You have installed the OpenShift Serverless Logic **kn-workflow** CLI plugin.
- You have created an OpenShift Serverless Logic workflow project.

Procedure

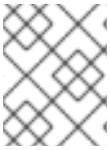
1. From the directory where you created your OpenShift Serverless Logic project, move to your project directory by running the following command:

```
$ cd ./<your-project-name>
```

2. Run the following command to build and run your OpenShift Serverless Logic workflow project:

```
$ kn workflow run
```

When the project is ready, the Development UI automatically opens in your browser at **localhost:8080/q/dev-ui** and you will find the **Serverless Workflow Tools** tile available. Alternatively, you can access the tool directly using **http://localhost:8080/q/dev-ui/org.apache.kie.sonataflow.sonataflow-quarkus-devui/workflows**.



NOTE

You can execute a workflow locally using a container that runs on your machine. Stop the container with Ctrl+C.

1.2. DEPLOYMENT OPTIONS AND DEPLOYING WORKFLOWS

You can deploy the Serverless Logic workflows on the cluster using one of three deployment profiles:

- Dev
- Preview
- GitOps

Each profile defines how the Operator builds and manages workflow deployments, including image lifecycle, live updates, and reconciliation behavior.

1.2.1. Deploying workflows using the Dev profile

You can deploy your local workflow on OpenShift Container Platform using the Dev profile. You can use this deployment to experiment and modify your workflow directly on the cluster, seeing changes almost immediately. Dev profile is designed for development and testing purposes. Because it automatically reloads the workflow without restarting the container, it is ideal for initial development stages and for testing workflow changes in a live environment.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Create the workflow configuration YAML file.

   Example workflow-dev.yaml file

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: greeting (1)
  annotations:
    sonataflow.org/description: Greeting example on k8s!
```

```
    sonataflow.org/version: 0.0.1
    sonataflow.org/profile: dev (2)
spec:
  flow:
    start: ChooseOnLanguage
    functions:
      - name: greetFunction
        type: custom
        operation: sysout
    states:
      - name: ChooseOnLanguage
        type: switch
        dataConditions:
          - condition: "${ .language == \"English\" }"
            transition: GreetInEnglish
          - condition: "${ .language == \"Spanish\" }"
            transition: GreetInSpanish
        defaultCondition: GreetInEnglish
      - name: GreetInEnglish
        type: inject
        data:
          greeting: "Hello from JSON Workflow, "
        transition: GreetPerson
      - name: GreetInSpanish
        type: inject
        data:
          greeting: "Saludos desde JSON Workflow, "
        transition: GreetPerson
      - name: GreetPerson
        type: operation
        actions:
          - name: greetAction
            functionRef:
              refName: greetFunction
              arguments:
                message:  ".greeting + .name"
        end: true
```

1. Is a <workflow_name>.
2. Indicates that you must deploy the workflow using the Dev profile.

2. To deploy the application, apply the YAML file by entering the following command:

```
$ oc apply -f <filename> -n <your_namespace>
```

3. Verify the deployment and check the status of the deployed workflow by entering the following command:

```
$ oc get workflow -n <your_namespace> -w
```

Ensure that your workflow is listed and the status is **Running** or **Completed**.

4. Edit the workflow directly in the cluster by entering the following command:

–

```
$ oc edit sonataflow <workflow_name> -n <your_namespace>
```

5. After editing, save the changes. The OpenShift Serverless Logic Operator detects the changes and updates the workflow accordingly.

Verification

1. To ensure the changes are applied correctly, verify the status and logs of the workflow by entering the following commands:
   a. View the status of the workflow by running the following command:

   ```
   $ oc get sonataflows -n <your_namespace>
   ```

   b. View the workflow logs by running the following command:

   ```
   $ oc logs <workflow_pod_name> -n <your_namespace>
   ```

Next steps

1. After completing the testing, delete the resources to avoid unnecessary usage by running the following command:

```
$ oc delete sonataflow <workflow_name> -n <your_namespace>
```

1.2.2. Deploying workflows using the Preview profile

You can deploy your local workflow on OpenShift Container Platform using the Preview profile. This allows you to validate and test workflows in a production-like environment directly on the cluster. Preview profile is ideal for final testing and validation before moving workflows to production, as well as for quick iteration without directly managing the build pipeline. It also ensures that workflows will run smoothly in a production-like setting.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

To deploy a workflow in Preview profile, OpenShift Serverless Logic Operator uses the build system on OpenShift Container Platform, which automatically creates the image for deploying your workflow.

The following sections explain how to build and deploy your workflow on a cluster using the OpenShift Serverless Logic Operator with a **SonataFlow** custom resource.

1.2.2.1. Configuring workflows in Preview profile

1.2.2.1.1. Configuring the workflow base builder image

If your scenario requires strict policies for image usage, such as security or hardening constraints, replace the default image used by the OpenShift Serverless Logic Operator to build the final workflow container image.

By default, the OpenShift Serverless Logic Operator uses the image distributed in the official Red Hat Registry to build workflows. If your scenario requires strict policies for image use, such as security or hardening constraints, you can replace the default image.

To change this image, you edit the **SonataFlowPlatform** custom resource (CR) in the namespace where you deployed your workflows.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. List the **SonataFlowPlatform** resources in your namespace by running the following command:

```
$ oc get sonataflowplatform -n <your_namespace> (1)
```

(1) Replace **<your_namespace>** with the name of your namespace.

2. Patch the **SonataFlowPlatform** resource with the new builder image by running the following command:

```
$ oc patch sonataflowplatform <name> --patch 'spec:\n  build:\n    config:\n      baseImage:
<your_new_image_full_name_with_tag>' -n <your_namespace>
```

Verification

1. Verify that the **SonataFlowPlatform** CR has been patched correctly by running the following command:

```
$ oc describe sonataflowplatform <name> -n <your_namespace> (1)
```

(1) Replace **<name>** with the name of your **SonataFlowPlatform** resource and **<your_namespace>** with the name of your namespace.

Ensure that the **baseImage** field under **spec.build.config** reflects the new image.

1.2.2.1.2. Customization for the base builder Dockerfile

The OpenShift Serverless Logic Operator uses the **logic-operator-rhel8-builder-config** config map custom resource (CR) in its **openshift-serverless-logic**OpenShift Serverless Logic Operator installation namespace to configure and run the workflow build process. You can change the Dockerfile entry in this config map to adjust the Dockerfile to your needs.

IMPORTANT

Modifying the Dockerfile can break the build process.

NOTE

This example is for reference only. The actual version might be slightly different. Do not use this example for your installation.

Example logic-operator-rhel8-builder-config config map CR

```
apiVersion: v1
data:
  DEFAULT_WORKFLOW_EXTENSION: .sw.json
  Dockerfile: |
    FROM registry.redhat.io/openshift-serverless-1/logic-swf-builder-rhel8:1.33.0 AS builder

    # Variables that can be overridden by the builder
    # To add a Quarkus extension to your application
    ARG QUARKUS_EXTENSIONS
    # Args to pass to the Quarkus CLI add extension command
    ARG QUARKUS_ADD_EXTENSION_ARGS
    # Additional java/mvn arguments to pass to the builder
    ARG MAVEN_ARGS_APPEND

    # Copy from build context to skeleton resources project
    COPY --chown=1001 . ./resources

    RUN /home/kogito/launch/build-app.sh ./resources

    #============================
    # Runtime Run
    #============================
    FROM registry.access.redhat.com/ubi9/openjdk-17:latest

    ENV LANG='en_US.UTF-8' LANGUAGE='en_US:en'

    # We make four distinct layers so if there are application changes, the library layers can be re-used
    COPY --from=builder --chown=185 /home/kogito/serverless-workflow-project/target/quarkus-app/lib/ /deployments/lib/
    COPY --from=builder --chown=185 /home/kogito/serverless-workflow-project/target/quarkus-app/*.jar /deployments/
    COPY --from=builder --chown=185 /home/kogito/serverless-workflow-project/target/quarkus-app/app/ /deployments/app/
    COPY --from=builder --chown=185 /home/kogito/serverless-workflow-project/target/quarkus-app/quarkus/ /deployments/quarkus/

    EXPOSE 8080
    USER 185
    ENV AB_JOLOKIA_OFF=""
    ENV JAVA_OPTS="-Dquarkus.http.host=0.0.0.0 -Djava.util.logging.manager=org.jboss.logmanager.LogManager"
    ENV JAVA_APP_JAR="/deployments/quarkus-run.jar"
kind: ConfigMap
```

```
metadata:
  name: sonataflow-operator-builder-config
  namespace: sonataflow-operator-system
```

1.2.2.1.3. Changing resource requirements

You can specify resource requirements for the internal builder pods, by creating or editing a **SonataFlowPlatform** resource in the workflow namespace.

Example SonataFlowPlatform resource

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform
spec:
  build:
    template:
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
```

NOTE

Only one **SonataFlowPlatform** resource is allowed per namespace. Fetch and edit the resource that the OpenShift Serverless Logic Operator created for you instead of trying to create another resource.

You can fine-tune the resource requirements for a particular workflow. Each workflow instance has a **SonataFlowBuild** instance created with the same name as the workflow. You can edit the **SonataFlowBuild** custom resource (CR) and specify the parameters as follows:

Example of SonataFlowBuild CR

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowBuild
metadata:
  name: my-workflow
spec:
  resources:
    requests:
      memory: "64Mi"
      cpu: "250m"
    limits:
      memory: "128Mi"
      cpu: "500m"
```

These parameters apply only to new build instances.

1.2.2.1.4. Passing arguments to the internal builder

You can customize the build process by passing build arguments to the **SonataFlowBuild** instance or setting default build arguments in the **SonataFlowPlatform** resource.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Check for the existing **SonataFlowBuild** instance by running the following command:

```
$ oc get sonataflowbuild <name> -n <namespace> (1)
```

(1) Replace **<name>** with the name of your **SonataFlowBuild** instance and **<namespace>** with your namespace.

2. Add build arguments to the **SonataFlowBuild** instance by running the following command:

```
$ oc edit sonataflowbuild <name> -n <namespace>
```

3. Add the desired build arguments under the **.spec.buildArgs** field of the **SonataFlowBuild** instance:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowBuild
metadata:
  name: <name> (1)
spec:
  buildArgs:
    - name: <argument_1>
      value: <value_1>
    - name: <argument_2>
      value: <value_2>
```

(1) The name of the existing **SonataFlowBuild** instance.

4. Save the file and exit.
   A new build with the updated configuration starts.
5. Set the default build arguments in the **SonataFlowPlatform** resource by running the following command:

```
$ oc edit sonataflowplatform <name> -n <namespace>
```

6. Add the desired build arguments under the **.spec.buildArgs** field of the **SonataFlowPlatform**

6. Add the desired build arguments under the **.spec.buildArgs** field of the **SonataFlowPlatform** resource:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: <name> (1)
spec:
  build:
    template:
      buildArgs:
        - name: <argument_1>
          value: <value_1>
        - name: <argument_2>
          value: <value_2>
```

(1) The name of the existing **SonataFlowPlatform** resource.

7. Save the file and exit.

1.2.2.1.5. Setting environment variables in the internal builder

You can set environment variables to the **SonataFlowBuild** internal builder pod. These variables are valid for the build context only and are not set on the final built workflow image.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Check for existing **SonataFlowBuild** instance by running the following command:

```
$ oc get sonataflowbuild <name> -n <namespace>
```

Replace **<name>** with the name of your **SonataFlowBuild** instance and **<namespace>** with your namespace.

2. Edit the **SonataFlowBuild** instance by running the following command:

```
$ oc edit sonataflowbuild <name> -n <namespace>
```

Example SonataFlowBuild instance

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowBuild
metadata:
  name: <name>
```

```
spec:
  envs:
    - name: <env_variable_1>
      value: <value_1>
    - name: <env_variable_2>
      value: <value_2>
```

3. Save the file and exit.
   A new with the updated configuration starts.

   Alternatively, you can set the enviroments in the **SonataFlowPlatform**, so that every new build instances will use it as a template.

   Example SonataFlowPlatform instance

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: <name>
spec:
  build:
    template:
      envs:
        - name: <env_variable_1>
          value: <value_1>
        - name: <env_variable_2>
          value: <value_2>
```

1.2.2.1.6. Changing the base builder image

You can modify the default builder image used by the OpenShift Serverless Logic Operator by editing the **logic-operator-rhel8-builder-config** config map.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Edit the **logic-operator-rhel8-builder-config** config map by running the following command:

```
$ oc edit cm/logic-operator-rhel8-builder-config -n openshift-serverless-logic
```

2. Modify the dockerfile entry.
   In your editor, locate the Dockerfile entry and change the first line to the desired image.

   Example

```
data:
```

```
Dockerfile: |
  FROM registry.redhat.io/openshift-serverless-1/logic-swf-builder-rhel8:1.33.0
  # Change the image to the desired one
```

3. Save the changes.

1.2.2.2. Building and deploying your workflow

You can create a **SonataFlow** custom resource (CR) on OpenShift Container Platform and OpenShift Serverless Logic Operator builds and deploys the workflow.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Create a workflow YAML file similar to the following:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: greeting
  annotations:
    sonataflow.org/description: Greeting example on k8s!
    sonataflow.org/version: 0.0.1
spec:
  flow:
    start: ChooseOnLanguage
    functions:
      - name: greetFunction
        type: custom
        operation: sysout
    states:
      - name: ChooseOnLanguage
        type: switch
        dataConditions:
          - condition: "${ .language == \"English\" }"
            transition: GreetInEnglish
          - condition: "${ .language == \"Spanish\" }"
            transition: GreetInSpanish
        defaultCondition: GreetInEnglish
      - name: GreetInEnglish
        type: inject
        data:
          greeting: "Hello from JSON Workflow, "
        transition: GreetPerson
      - name: GreetInSpanish
        type: inject
        data:
```

```
        greeting: "Saludos desde JSON Workflow, "
      transition: GreetPerson
    - name: GreetPerson
      type: operation
      actions:
        - name: greetAction
          functionRef:
            refName: greetFunction
            arguments:
              message:  ".greeting+.name"
      end: true
```

2. Apply the **SonataFlow** workflow definition to your OpenShift Container Platform namespace by running the following command:

```
$ oc apply -f <workflow-name>.yaml -n <your_namespace>
```

Example command for the greetings-workflow.yaml file:

```
$ oc apply -f greetings-workflow.yaml -n workflows
```

3. List all the build configurations by running the following command:

```
$ oc get buildconfigs -n workflows
```

4. Get the logs of the build process by running the following command:

```
$ oc logs buildconfig/<workflow-name> -n <your_namespace>
```

Example command for the greetings-workflow.yaml file:

```
$ oc logs buildconfig/greeting -n workflows
```

Verification

1. To verify the deployment, list all the pods by running the following command:

```
$ oc get pods -n <your_namespace>
```

Ensure that the pod corresponding to your workflow is running.

2. Check the running pods and their logs by running the following command:

```
$ oc logs pod/<pod-name> -n workflows
```

1.2.2.3. Verifying workflow deployment

You can verify that your OpenShift Serverless Logic workflow is running by performing a test HTTP call from the workflow pod.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Create a workflow **YAML** file similar to the following:

```yaml
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: greeting
  annotations:
    sonataflow.org/description: Greeting example on k8s!
    sonataflow.org/version: 0.0.1
spec:
  flow:
    start: ChooseOnLanguage
    functions:
      - name: greetFunction
        type: custom
        operation: sysout
    states:
      - name: ChooseOnLanguage
        type: switch
        dataConditions:
          - condition: "${ .language == \"English\" }"
            transition: GreetInEnglish
          - condition: "${ .language == \"Spanish\" }"
            transition: GreetInSpanish
        defaultCondition: GreetInEnglish
      - name: GreetInEnglish
        type: inject
        data:
          greeting: "Hello from JSON Workflow, "
        transition: GreetPerson
      - name: GreetInSpanish
        type: inject
        data:
          greeting: "Saludos desde JSON Workflow, "
        transition: GreetPerson
      - name: GreetPerson
        type: operation
        actions:
          - name: greetAction
            functionRef:
              refName: greetFunction
              arguments:
                message:  ".greeting+.name"
        end: true
```

2. Create a route for the workflow service by running the following command:

```
$ oc expose svc/<workflow-service-name> -n workflows
```

This command creates a public URL to access the workflow service.

3. Set an environment variable for the public URL by running the following command:

```
$ WORKFLOW_SVC=$(oc get route/<workflow-service-name> -n <namespace> --
template='{{.spec.host}}')
```

4. Make an HTTP call to the workflow to send a POST request to the service by running the following command:

```
$ curl -X POST -H 'Content-Type: application/json' -H 'Accept: application/json' -d '{<"your":
"json_payload">}' http://$WORKFLOW_SVC/<endpoint>
```

Example output

```
{
  "id": "b5fbfaa3-b125-4e6c-9311-fe5a3577efdd",
  "workflowdata": {
    "name": "John",
    "language": "English",
    "greeting": "Hello from JSON Workflow, "
  }
}
```

This output shows an example of the expected response if the workflow is running.

1.2.2.4. Restarting a build

To restart a build, you can add or edit the **sonataflow.org/restartBuild: true** annotation in the **SonataFlowBuild** instance. Restarting a build is necessary if there is a problem with your workflow or the initial build revision.

Prerequisites

- You have an OpenShift Serverless Logic Operator installed on your cluster.
- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Check if the **SonataFlowBuild** instance exists by running the following command:

```
$ oc get sonataflowbuild <name> -n <namespace>
```

2. Edit the **SonataFlowBuild** instance by running the following command:

```
$ oc edit sonataflowbuild/<name> -n <namespace>
```

Replace **<name>** with the name of your **SonataFlowBuild** instance and **<namespace>** with the namespace where your workflow is deployed.

3. Add the **sonataflow.org/restartBuild: true** annotation to restart the build.

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowBuild
metadata:
  name: <name>
  annotations:
    sonataflow.org/restartBuild: true
```

This action triggers the OpenShift Serverless Logic Operator to start a new build of the workflow.

4. To monitor the build process, check the build logs by running the following command:

```
$ oc logs buildconfig/<name> -n <namespace>
```

Replace **<name>** with the name of your **SonataFlowBuild** instance and **<namespace>** with the namespace where your workflow is deployed.

1.2.3. Deploying workflows using the GitOps profile

IMPORTANT

Use the GitOps profile only for production deployments. For development, rapid iteration, or testing, use the Dev or Preview profiles instead.

You can deploy your local workflow on OpenShift Container Platform using the GitOps profile. The GitOps profile provides full control over the workflow container image by allowing you to build and manage the image externally, typically through a CI/CD pipeline such as ArgoCD or Tekton. When a container image is defined in the **SonataFlow** custom resource (CR), the Operator automatically assumes the GitOps profile is being used and it does not attempt to build or modify the image in any way.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Specify your container image in your SonataFlow CR:

Example SonataFlow CR with set GitOps profile

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
```

```
  annotations:
    sonataflow.org/profile: gitops
  name: workflow_name
spec:
  flow: (1)
#...
  podTemplate:
    container:
      image: your-registry/workflow_name:tag
#...
```

(1) The **flow** definition must match the workflow definition used during the build process. When you deploy your workflow using the GitOps profile, the Operator compares this definition with the workflow files embedded in the container image. If the definition and files do not match, the deployment fails.

2. Apply your CR to deploy the workflow:

```
$ oc apply -f <filename>
```

1.2.4. Editing a workflow

When the OpenShift Serverless Logic Operator deploys a workflow service, it creates two config maps to store runtime properties:

- User properties: Defined in a **ConfigMap** named after the **SonataFlow** object with the suffix **-props**. For example, if your workflow name is **greeting**, then the **ConfigMap** name is **greeting-props**.
- Managed properties: Defined in a **ConfigMap** named after the **SonataFlow** object with the suffix **-managed-props**. For example, if your workflow name is **greeting**, then the **ConfigMap** name is **greeting-managed-props**.

> **NOTE**
>
> Managed properties always override any user property with the same key name and cannot be edited by the user. Any change would be overwritten by the Operator at the next reconciliation cycle.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Open and edit the **ConfigMap** by running the following command:

```
$ oc edit cm <workflow_name>-props -n <namespace>
```

–

Replace **<workflow_name>** with the name of your workflow and **<namespace>** with the namespace where your workflow is deployed.

2. Add the properties in the **application.properties** section.

Example of a workflow properties stored within a ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app: greeting
  name: greeting-props
  namespace: default
data:
  application.properties: |
    my.properties.key = any-value
```

Ensure the properties are correctly formatted to prevent the Operator from replacing your configuration with the default one.

3. After making the necessary changes, save the file and exit the editor.

1.2.5. Testing a workflow

To verify that your OpenShift Serverless Logic workflow is running correctly, you can perform a test HTTP call from the relevant pod.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. To create a route for the specified service in your namespace by running the following command:

```
$ oc expose svc <service_name> -n <namespace>
```

2. To fetch the URL for the newly exposed service by running the following command:

```
$ WORKFLOW_SVC=$(oc get route/<service_name> --template='{{.spec.host}}')
```

3. Perform a test HTTP call and send a **POST** request by running the following command:

```
$ curl -X POST -H 'Content-Type:application/json' -H 'Accept:application/json' -d
'<request_body>' http://$WORKFLOW_SVC/<endpoint>
```

4. Verify the response to ensure the workflow is functioning as expected.

1.2.6. Troubleshooting a workflow

The OpenShift Serverless Logic Operator deploys its pod with health check probes to ensure the Workflow runs in a healthy state. If changes cause these health checks to fail, the pod will stop responding.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Check the workflow status by running the following command:

```
$ oc get workflow <name> -o jsonpath={.status.conditions} | jq .
```

2. To fetch and analyze the the logs from the workflow's deployment, run the following command:

```
$ oc logs deployment/<workflow_name> -f
```

1.2.7. Deleting a workflow

You can use the **oc delete** command to delete your OpenShift Serverless Logic workflow in your current directory.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have installed the OpenShift CLI **(oc)**.

Procedure

1. Verify that you have the correct file that defines the Workflow you want to delete. For example, **workflow.yaml**.
2. Run the **oc delete** command to remove the Workflow from your specified namespace:

```
$ oc delete -f <your_file> -n <your_namespace>
```

   Replace **<your_file>** with the name of your Workflow file and **<your_namespace>** with your namespace.

CHAPTER 2. GLOBAL CONFIGURATION SETTINGS

You can set global configuration options for the OpenShift Serverless Logic Operator.

2.1. PREREQUISITES

- You have installed the OpenShift Serverless Logic Operator in the target cluster.

2.2. CUSTOMIZATION OF GLOBAL CONFIGURATIONS

After installing the OpenShift Serverless Logic Operator, you can access the **logic-operator-rhel8-controllers-config** config map file in the **openshift-serverless-logic** namespace. This configuration file defines how the Operator behaves when it creates new resources in the cluster. However, changes to this configuration does not affect resources that already exist.

You can modify any of the options within the **controllers_cfg.yaml** key in the config map.

The following table outlines all the available global configuration options:

Configuration key	Default value	Description
defaultPvcKanikoSize	**1Gi**	The default size of Kaniko persistent volume claim (PVC) when using the internal OpenShift Serverless Logic Operator builder manager.
healthFailureThresholdDevMode	**50**	How much time (in seconds) to wait for a developer mode workflow to start. This information is used for the controller manager to create new developer mode containers and setup the health check probes.
kanikoDefaultWarmerImageTag	**gcr.io/kaniko-project/warmer:v1.9.0**	Default image used internally by the Operator managed Kaniko builder to create the warmup pods.
kanikoExecutorImageTag	**gcr.io/kaniko-project/executor:v1.9.0**	Default image used internally by the Operator managed Kaniko builder to create the executor pods.
jobsServicePostgreSQLImageTag	**registry.redhat.io/openshift-serverless-1/logic-jobs-service-postgresql-rhel8:1.35.0**	The Jobs service image for PostgreSQL to use. If empty, the OpenShift Serverless Logic Operator uses the default Apache community image based on the current OpenShift Serverless Logic Operator version.

Configuration key	Default value	Description
jobsServiceEphemeralImageTag	**registry.redhat.io/openshift-serverless-1/logic-jobs-service-ephemeral-rhel8:1.35.0**	The Jobs service image without persistence to use. If empty, the OpenShift Serverless Logic Operator uses the default Apache community image based on the current OpenShift Serverless Logic Operator version.
dataIndexPostgreSQLImageTag	**registry.redhat.io/openshift-serverless-1/logic-data-index-postgresql-rhel8:1.35.0**	The Data Index service image for PostgreSQL to use. If empty, the OpenShift Serverless Logic Operator uses the default Apache community image based on the current OpenShift Serverless Logic Operator version.
dataIndexEphemeralImageTag	**registry.redhat.io/openshift-serverless-1/logic-data-index-ephemeral-rhel8:1.35.0**	The Data Index service image without persistence to use. If empty, the OpenShift Serverless Logic Operator uses the default Apache community image based on the current OpenShift Serverless Logic Operator version.
sonataFlowBaseBuilderImageTag	**registry.redhat.io/openshift-serverless-1/logic-swf-builder-rhel8:1.35.0**	OpenShift Serverless Logic base builder image used in the internal Dockerfile to build workflow applications in preview profile. If empty, the OpenShift Serverless Logic Operator uses the default Apache community image based on the current OpenShift Serverless Logic Operator version.

Configuration key	Default value	Description
sonataFlowDevModeImageTag	**registry.redhat.io/openshift-serverless-1/logic-swf-devmode-rhel8:1.35.0**	The image to use to deploy OpenShift Serverless Logic workflow images in devmode profile. If empty, the OpenShift Serverless Logic Operator uses the default Apache community image based on the current OpenShift Serverless Logic Operator version.
builderConfigMapName	**logic-operator-rhel8-builder-config**	The default name of the builder config map in the OpenShift Serverless Logic Operator namespace.
postgreSQLPersistenceExtensions	next column	Quarkus extensions required for workflows persistence. These extensions are used by the OpenShift Serverless Logic Operator builder in cases where the workflow being built has configured PostgreSQL persistence.
kogitoEventsGrouping	**true**	When set to **true**, configures every workflow deployment with the **gitops** or **preview** profiles to send accumulated workflow status change events to the Data Index service, reducing the number of produced events. You can set the value to **false** to send individual events.
kogitoEventsGroupingBinary	**true**	When set to **true**, the accumulated workflow status change events are sent in binary mode, reducing the size of the produced events. You can set the value to **false** to send plain JSON events.
kogitoEventsGroupingCompress	**false**	When set to **true**, the accumulated workflow status change events, if sent in binary mode, are zipped at the cost of some performance.

You can edit this by updating the **logic-operator-controllers-config** config map by using the **oc** command-line tool.

2.2.1. Impact of global configuration changes

When you update the global configurations, the changes immediately affect only newly created resources. For example, if you change the **sonataFlowDevModeImageTag** property and already have a workflow deployed in dev mode, the OpenShift Serverless Logic Operator does not roll out a new deployment with the updated image configuration. Only new deployments reflect the changes.

2.2.2. Customizing the base builder image

You can directly change the base builder image in the Dockerfile used by the OpenShift Serverless Logic Operator.

Additionally, you can specify the base builder image in the **SonataFlowPlatform** configuration within the current namespace. This ensures that the specified base image is used exclusively in the given namespace.

Example of SonataFlowPlatform with a custom base builder image

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform
spec:
  build:
    config:
        baseImage: dev.local/my-workflow-builder:1.0.0
```

Alternatively, you can also modify the base builder image in the global configuration config map as shown in the following example:

Example of ConfigMap with a custom base builder image

```
apiVersion: v1
data:
  controllers_cfg.yaml: |
    sonataFlowBaseBuilderImageTag: dev.local/my-workflow-builder:1.0.0
kind: ConfigMap
metadata:
  name: logic-operator-rhel8-controllers-config
  namespace: openshift-serverless-logic
```

When customizing the base builder image, the following order of precedence is applicable:

1. The **SonataFlowPlatform** configuration in the current context.
2. The global configuration entry in the **ConfigMap** resource.
3. The **FROM** clause in the Dockerfile within the OpenShift Serverless Logic Operator namespace, defined in the **logic-operator-rhel8-builder-config** config map.

The entry in the **SonataFlowPlatform** configuration always overrides any other value.

CHAPTER 3. MANAGING SERVICES

3.1. CONFIGURING OPENAPI SERVICES

The *OpenAPI Specification* (OAS) defines a standard, programming language-agnostic interface for HTTP APIs. You can understand a service's capabilities without access to the source code, additional documentation, or network traffic inspection. When you define a service by using the OpenAPI, you can understand and interact with it using minimal implementation logic. Just as interface descriptions simplify lower-level programming, the *OpenAPI Specification* eliminates guesswork in calling a service.

3.1.1. OpenAPI function definition

OpenShift Serverless Logic allows the workflows to interact with remote services using an OpenAPI specfication reference in a function.

Example OpenAPI function definition

```
{
  "functions": [
    {
      "name": "myFunction1",
      "operation": "specs/myopenapi-file.yaml#myFunction1"
    }
  ]
}
```

The **operation** attribute is a **string** composed of the following parameters:

- **URI**: The engine uses this to locate the specification file.
- Operation identifier: You can find this identifier in the OpenAPI specification file.

OpenShift Serverless Logic supports the following URI schemes:

- file: Use this for files located in the file system.
- **http** or **https**: Use these for remotely located files.

Ensure the OpenAPI specification files are available during build time. OpenShift Serverless Logic uses an internal code generation feature to send requests at runtime. After you build the application image, OpenShift Serverless Logic will not have access to these files.

If the OpenAPI service you want to add to the workflow does not have a specification file, you can either create one or update the service to generate and expose the file.

3.1.2. Sending REST requests based on the OpenAPI specification

To send REST requests that are based on the OpenAPI specification files, you must perform the following procedures:

- Define the function references
- Access the defined functions in the workflow states

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to the OpenAPI specification files.

Procedure

1. To define the OpenAPI functions:
   a. Identify and access the OpenAPI specification files for the services you intend to invoke.
   b. Copy the OpenAPI specification files into your workflow service directory, such as **<project_application_dir>/specs**.
   The following example shows the OpenAPI specification for the multiplication REST service:

Example multiplication REST service OpenAPI specification

```
openapi: 3.0.3
info:
  title: Generated API
  version: "1.0"
paths:
  /:
    post:
      operationId: doOperation
      parameters:
        - in: header
          name: notUsed
          schema:
            type: string
          required: false
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/MultiplicationOperation'
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema:
                type: object
                properties:
                  product:
                    format: float
                    type: number
components:
  schemas:
    MultiplicationOperation:
      type: object
      properties:
```

```
        leftElement:
          format: float
          type: number
        rightElement:
          format: float
          type: number
```

c. To define functions in the workflow, use the **operationId** from the OpenAPI specification to reference the desired operations in your function definitions.

Example function definitions in the temperature conversion application

```
{
  "functions": [
    {
      "name": "multiplication",
      "operation": "specs/multiplication.yaml#doOperation"
    },
    {
      "name": "subtraction",
      "operation": "specs/subtraction.yaml#doOperation"
    }
  ]
}
```

d. Ensure that your function definitions reference the correct paths to the OpenAPI files stored in the **<project_application_dir>/specs** directory.

2. To access the defined functions in the workflow states:

a. Define workflow actions to call the function definitions you added. Ensure each action references a function defined earlier.

b. Use the **functionRef** attribute to refer to the specific function by its name. Map the arguments in the **functionRef** using the parameters defined in the OpenAPI specification. The following example shows about mapping parameters in the request path instead of request body, you can refer to the following PetStore API example:

Example for mapping function arguments in workflow

```
{
  "states": [
    {
      "name": "SetConstants",
      "type": "inject",
      "data": {
        "subtractValue": 32.0,
        "multiplyValue": 0.5556
      },
      "transition": "Computation"
    },
    {
      "name": "Computation",
      "actionMode": "sequential",
      "type": "operation",
```

```
      "actions": [
        {
          "name": "subtract",
          "functionRef": {
            "refName": "subtraction",
            "arguments": {
              "leftElement": ".fahrenheit",
              "rightElement": ".subtractValue"
            }
          }
        },
        {
          "name": "multiply",
          "functionRef": {
            "refName": "multiplication",
            "arguments": {
              "leftElement": ".difference",
              "rightElement": ".multiplyValue"
            }
          }
        }
      ],
      "end": {
        "terminate": true
      }
    }
  ]
}
```

c. Check the **Operation Object** section of the OpenAPI specification to understand how to structure parameters in the request.

d. Use **jq** expressions to extract data from the payload and map it to the required parameters. Ensure the engine maps parameter names according to the OpenAPI specification.

e. For operations requiring parameters in the request path instead of the body, refer to the parameter definitions in the OpenAPI specification.
For more information about mapping parameters in the request path instead of request body, you can refer to the following PetStore API example:

Example for mapping path parameters

```
{
  "/pet/{petId}": {
    "get": {
      "tags": ["pet"],
      "summary": "Find pet by ID",
      "description": "Returns a single pet",
      "operationId": "getPetById",
      "parameters": [
        {
          "name": "petId",
          "in": "path",
          "description": "ID of pet to return",
          "required": true,
          "schema": {
```

```
            "type": "integer",
            "format": "int64"
          }
        }
      ]
    }
  }
}
```

Following is an example invocation of a function, in which only one parameter named **petId** is added in the request path:

Example of calling the PetStore function

```
{
  "name": "CallPetStore", (1)
  "actionMode": "sequential",
  "type": "operation",
  "actions": [
    {
      "name": "getPet",
      "functionRef": {
        "refName": "getPetById", (2)
        "arguments": { (3)
          "petId": ".petId"
        }
      }
    }
  ]
}
```

1. State definition, such as **CallPetStore**.
2. Function definition reference. In the previous example, the function definition **getPetById** is for PetStore OpenAPI specification.
3. Arguments definition. OpenShift Serverless Logic adds the argument **petId** to the request path before sending a request.

3.1.3. Configuring the endpoint URL of OpenAPI services

After accessing the function definitions in workflow states, you can configure the endpoint URL of OpenAPI services.

Prerequisites

- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have created your OpenShift Serverless Logic project.
- You have access to the OpenAPI specification files.
- You have defined the function definitions in the workflow.

- You have the access to the defined functions in the workflow states.

Procedure

1. Locate the OpenAPI specification file you want to configure. For example, **substraction.yaml**.
2. Convert the file name into a valid configuration key by replacing special characters, such as **.**, with underscores and converting letters to lowercase. For example, change **substraction.yaml** to **substraction_yaml**.
3. To define the configuration key, use the converted file name as the REST client configuration key. Set this key as an environment variable, as shown in the following example:

```
quarkus.rest-client.subtraction_yaml.url=http://myserver.com
```

4. To prevent hardcoding URLs in the **application.properties** file, use environment variable substitution, as shown in the following example:

```
quarkus.rest-client.subtraction_yaml.url=${SUBTRACTION_URL:http://myserver.com}
```

   In this example:

   - Configuration Key: **quarkus.rest-client.subtraction_yaml.url**
   - Environment variable: SUBTRACTION_URL
   - Fallback URL: **http://myserver.com**

5. Ensure that the **(SUBTRACTION_URL)** environment variable is set in your system or deployment environment. If the variable is not found, the application uses the fallback URL **(http://myserver.com)**.
6. Add the configuration key and URL substitution to the **application.properties** file:

```
quarkus.rest-client.subtraction_yaml.url=${SUBTRACTION_URL:http://myserver.com}
```

7. Deploy or restart your application to apply the new configuration settings.

3.2. CONFIGURING OPENAPI SERVICES ENDPOINTS

OpenShift Serverless Logic uses the **kogito.sw.operationIdStrategy** property to generate the REST client for invoking services defined in OpenAPI documents. This property determines how the configuration key is derived for the REST client configuration.

The **kogito.sw.operationIdStrategy** property supports the following values: **FILE_NAME**, **FULL_URI**, **FUNCTION_NAME**, and **SPEC_TITLE**.

FILE_NAME

OpenShift Serverless Logic uses the OpenAPI document file name to create the configuration key. The key is based on the file name, where special characters are replaced with underscores.

Example configuration:

```
quarkus.rest-client.stock_portfolio_svc_yaml.url=http://localhost:8282/ (1)
```

1. The OpenAPI file path is **<project_application_dir>/specs/stock-portfolio-svc.yaml**. The generated key that configures the URL for the REST client is **stock_portfolio_svc_yaml**.

FULL_URI

OpenShift Serverless Logic uses the complete URI path of the OpenAPI document as the configuration key. The full URI is sanitized to form the key.

Example for Serverless Workflow

```
{
  "id": "myworkflow",
  "functions": [
    {
      "name": "myfunction",
      "operation": "https://my.remote.host/apicatalog/apis/123/document"
    }
  ]
}
```

Example configuration:

```
quarkus.rest-client.apicatalog_apis_123_document.url=http://localhost:8282/ (1)
```

1. The URI path is **https://my.remote.host/apicatalog/apis/123/document**. The generated key that configures the URL for the REST client is **apicatalog_apis_123_document**.

FUNCTION_NAME

OpenShift Serverless Logic combines the workflow ID and the function name referencing the OpenAPI document to generate the configuration key.

Example for Serverless Workflow

```
{
  "id": "myworkflow",
  "functions": [
    {
      "name": "myfunction",
      "operation": "https://my.remote.host/apicatalog/apis/123/document"
    }
  ]
}
```

Example configuration:

```
quarkus.rest-client.myworkflow_myfunction.url=http://localhost:8282/ (1)
```

1. The workflow ID is **myworkflow**. The function name is **myfunction**. The generated key that configures the URL for the REST client is **myworkflow_myfunction**.

SPEC_TITLE

OpenShift Serverless Logic uses the **info.title** value from the OpenAPI document to create the configuration key. The title is sanitized to form the key.

Example for OpenAPI document

```
openapi: 3.0.3
info:
  title: stock-service API
  version: 2.0.0-SNAPSHOT
paths:
  /stock-price/{symbol}:
...
```

Example configuration:

```
quarkus.rest-client.stock-service_API.url=http://localhost:8282/ (1)
```

1. The OpenAPI document title is **stock-service API**. The generated key that configures the URL for the REST client is **stock-service_API**.

3.2.1. Using URI alias

As an alternative to the **kogito.sw.operationIdStrategy** property, you can assign an alias to a URI by using the **workflow-uri-definitions** custom extension. This alias simplifies the configuration process and can be used as a configuration key in REST client settings and function definitions.

The **workflow-uri-definitions** extension allows you to map a URI to an alias, which you can reference throughout the workflow and in your configuration files. This approach provides a centralized way to manage URIs and their configurations.

Prerequisites

- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to the OpenAPI specification files.

Procedure

1. Add the **workflow-uri-definitions** extension to your workflow. Within this extension, create aliases for your URIs.

   Example workflow

```
{
  "extensions": [
```

```
    {
      "extensionid": "workflow-uri-definitions", (1)
      "definitions": {
        "remoteCatalog": "https://my.remote.host/apicatalog/apis/123/document" (2)
      }
    }
  ],
  "functions": [ (3)
    {
      "name": "operation1",
      "operation": "remoteCatalog#operation1"
    },
    {
      "name": "operation2",
      "operation": "remoteCatalog#operation2"
    }
  ]
}
```

1. Set the extension ID to **workflow-uri-definitions**.
2. Set the alias definition by mapping the **remoteCatalog** alias to a URI, for example, **https://my.remote.host/apicatalog/apis/123/document** URI.
3. Set the function operations by using the **remoteCatalog** alias with the operation identifiers, for example, **operation1** and **operation2** operation identifiers.

1. In the **application.properties** file, configure the REST client by using the alias defined in the workflow.

   Example property

   ```
   quarkus.rest-client.remoteCatalog.url=http://localhost:8282/
   ```

   In the previous example, the configuration key is set to **quarkus.rest-client.remoteCatalog.url**, and the URL is set to **http://localhost:8282/**, which the REST clients use by referring to the **remoteCatalog** alias.

2. In your workflow, use the alias when defining functions that operate on the URI.

   Example Workflow (continued):

   ```
   {
     "functions": [
       {
         "name": "operation1",
         "operation": "remoteCatalog#operation1"
       },
       {
         "name": "operation2",
         "operation": "remoteCatalog#operation2"
       }
     ]
   }
   ```

3.3. TROUBLESHOOTING SERVICES

Efficient troubleshooting of the HTTP-based function invocations, such as those using OpenAPI functions, is crucial for maintaining workflow orchestrations.

To diagnose issues, you can trace HTTP requests and responses.

3.3.1. Tracing HTTP requests and responses

OpenShift Serverless Logic uses the Apache HTTP client to the trace HTTP requests and responses.

Prerequisites

- You have access to an OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to the OpenAPI specification files.
- You have access to the workflow definition and instance IDs for correlating HTTP requests and responses.
- You have access to the log configuration of the application where the HTTP service invocations are occurring

Procedure

1. To trace HTTP requests and responses, OpenShift Serverless Logic uses the Apache HTTP client by setting the following property:

```
# Turning HTTP tracing on
quarkus.log.category."org.apache.http".level=DEBUG
```

2. Add the following configuration to your application's **application.properties** file to turn on debugging for the Apache HTTP Client:

```
quarkus.log.category."org.apache.http".level=DEBUG
```

3. Restart your application to propagate the log configuration changes.
4. After restarting, check the logs for HTTP request traces.

Example logs of a traced HTTP request

```
2023-09-25 19:00:55,242 DEBUG Executing request POST /v2/models/yolo-model/infer
HTTP/1.1
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> POST /v2/models/yolo-model/infer
HTTP/1.1
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> Accept: application/json
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> Content-Type: application/json
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> kogitoprocid: inferencepipeline
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> kogitoprocinstanceid: 85114b2d-9f64-
496a-bf1d-d3a0760cde8e
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> kogitoprocist: Active
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> kogitoproctype: SW
```

```
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> kogitoprocversion: 1.0
2023-09-25 19:00:55,243 DEBUG http-outgoing-0 >> Content-Length: 23177723
2023-09-25 19:00:55,244 DEBUG http-outgoing-0 >> Host: yolo-model-opendatahub-
model.apps.trustyai.dzzt.p1.openshiftapps.com
```

5. Check the logs for HTTP response traces following the request logs.

Example logs of a traced HTTP response

```
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "HTTP/1.1 500 Internal Server Error[\r]
[\n]"
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "content-type: application/json[\r][\n]"
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "date: Mon, 25 Sep 2023 19:01:00
GMT[\r][\n]"
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "content-length: 186[\r][\n]"
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "set-cookie:
276e4597d7fcb3b2cba7b5f037eeacf5=5427fafade21f8e7a4ee1fa6c221cf40; path=/;
HttpOnly; Secure; SameSite=None[\r][\n]"
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "[\r][\n]"
2023-09-25 19:01:00,738 DEBUG http-outgoing-0 << "{"code":13, "message":"Failed to load
Model due to adapter error: Error calling stat on model file: stat /models/yolo-model__isvc-
1295fd6ba9/yolov5s-seg.onnx: no such file or directory"}"
```

CHAPTER 4. SUPPORTING SERVICES

4.1. JOB SERVICE

The Job service schedules and executes tasks in a cloud environment. Independent services implement these tasks, which can be initiated through any of the supported interaction modes, including HTTP calls or Knative Events delivery.

In OpenShift Serverless Logic, the Job service is responsible for controlling the execution of the time-triggered actions. Therefore, all the time-based states that you can use in a workflow, are handled by the interaction between the workflow and the Job service.

For example, every time the workflow execution reaches a state with a configured timeout, a corresponding job is created in the Job service, and when the timeout is met, an HTTP callback is executed to notify the workflow.

The main goal of the Job service is to manage active jobs, such as scheduled jobs that need to be executed. When a job reaches its final state, the Job service removes it. To retain jobs information in a permanent repository, the Job service produces status change events that can be recorded by an external service, such as the Data Index Service.

NOTE

You do not need to manually install or configure the Job service if you are using the OpenShift Serverless Operator to deploy workflows. The Operator handles these tasks automatically and manages all necessary configurations for each workflow to connect with it.

4.1.1. Job service leader election process

The Job service operates as a singleton service, meaning only one active instance can schedule and execute jobs.

To prevent conflicts when the service is deployed in the cloud, where multiple instances might be running, the Job service supports a leader election process. Only the instance that is elected as the leader manages external communication to receive and schedule jobs.

Non-leader instances remain inactive in a standby state but continue attempting to become the leader through the election process. When a new instance starts, it does not immediately assume leadership. Instead, it enters the leader election process to determine if it can take over the leader role.

If the current leader becomes unresponsive or if it is shut down, another running instance takes over as the leader.

NOTE

This leader election mechanism uses the underlying persistence backend, which is currently supported only in the PostgreSQL implementation.

4.2. DATA INDEX SERVICE

The Data Index service is a dedicated supporting service that stores the data related to the workflow instances and their associated jobs. This service provides a GraphQL endpoint allowing users to query that data.

The Data Index service processes data received through events, which can originate from any workflow or directly from the Job service.

Data Index supports Apache Kafka or Knative Eventing to consume CloudEvents messages from workflows. It indexes and stores this event data in a database, making it accessible through GraphQL. These events provide detailed information about the workflow execution. The Data Index service is central to OpenShift Serverless Logic search, insights, and management capabilities.

The key features of the Data Index service are as follows:

- A flexible data structure
- A distributable, cloud-ready format
- Message-based communication with workflows via Apache Kafka, Knative, and CloudEvents
- A powerful GraphQL-based querying API

NOTE

When you are using the OpenShift Serverless Operator to deploy workflows, you do not need to manually install or configure the Data Index service. The Operator automatically manages all the necessary configurations for each workflow to connect with it.

4.2.1. GraphQL queries for workflow instances and jobs

To retrieve data about workflow instances and jobs, you can use GraphQL queries.

4.2.1.1. Retrieve data from workflow instances

You can retrieve information about a specific workflow instance by using the following query example:

```
{
  ProcessInstances {
    id
    processId
    state
    parentProcessInstanceId
    rootProcessId
    rootProcessInstanceId
    variables
    nodes {
      id
      name
      type
    }
  }
}
```

4.2.1.2. Retrieve data from jobs

You can retrieve data from a specific job instance by using the following query example:

```
{
```

```
  Jobs {
    id
    status
    priority
    processId
    processInstanceId
    executionCounter
  }
}
```

4.2.1.3. Filter query results by using the where parameter

You can filter query results by using the **where** parameter, allowing multiple combinations based on workflow attributes.

Example query to filter by state

```
{
  ProcessInstances(where: {state: {equal: ACTIVE}}) {
    id
    processId
    processName
    start
    state
    variables
  }
}
```

Example query to filter by ID

```
{
  ProcessInstances(where: {id: {equal: "d43a56b6-fb11-4066-b689-d70386b9a375"}}) {
    id
    processId
    processName
    start
    state
    variables
  }
}
```

By default, filters are combined using the AND Operator. You can modify this behavior by combining filters with the AND or OR operators.

Example query to combine filters with the OR Operator

```
{
  ProcessInstances(where: {or: {state: {equal: ACTIVE}, rootProcessId: {isNull: false}}}) {
    id
    processId
    processName
    start
    end
```

```
    state
  }
}
```

Example query to combine filters with the AND and OR Operators

```
{
  ProcessInstances(where: {and: {processId: {equal: "travels"}, or: {state: {equal: ACTIVE},
rootProcessId: {isNull: false}}}}) {
    id
    processId
    processName
    start
    end
    state
  }
}
```

Depending on the attribute type, you can use the following avaialable Operators:

Attribute type	Available Operators
String array	• **contains**: String • **containsAll**: Array of strings • **containsAny**: Array of strings • **isNull**: Boolean (true or false)
String	• **in**: Array of strings • **like**: String • **isNull**: Boolean (true or false) • **equal**: String
ID	• **in**: Array of strings • **isNull**: Boolean (true or false) • **equal**: String
Boolean	• **isNull**: Boolean (true or false) • **equal**: Boolean (true or false)

Attribute type	Available Operators
Numeric	• **in**: Array of integers • **isNull**: Boolean • **equal**: Integer • **greaterThan**: Integer • **greaterThanEqual**: Integer • **lessThan**: Integer • **lessThanEqual**: Integer • **between**: Numeric range • **from**: Integer • **to**: Integer
Date	• **isNull**: Boolean (true or false) • **equal**: Date time • **greaterThan**: Date time • **greaterThanEqual**: Date time • **lessThan**: Date time • **lessThanEqual**: Date time • **between**: Date range • **from**: Date time • **to**: Date time

4.2.1.4. Sort query results by using the orderBy parameter

You can sort query results based on workflow attributes by using the **orderBy** parameter. You can also specify the sorting direction in an ascending (**ASC**) or a descending (**DESC**) order. Multiple attributes are applied in the order you specified.

Example query to sort by the start time in an ASC order

```
{
  ProcessInstances(where: {state: {equal: ACTIVE}}, orderBy: {start: ASC}) {
    id
    processId
    processName
    start
    end
```

```
    state
  }
}
```

4.2.1.5. Limit the number of results by using the pagination parameter

You can control the number of returned results and specify an offset by using the **pagination** parameter.

Example query to limit results to 10, starting from offset 0

```
{
  ProcessInstances(where: {state: {equal: ACTIVE}}, orderBy: {start: ASC}, pagination: {limit: 10,
offset: 0}) {
    id
    processId
    processName
    start
    end
    state
  }
}
```

4.3. MANAGING SUPPORTING SERVICES

This section provides an overview of the supporting services essential for OpenShift Serverless Logic. It specifically focuses on configuring and deploying the Data Index service and Job Service supporting services using the OpenShift Serverless Logic Operator.

In a typical OpenShift Serverless Logic installation, you must deploy both services to ensure successful workflow execution. The Data Index service allows for efficient data management, while the Job Service ensures reliable job handling.

4.3.1. Supporting services and workflow integration

When you deploy a supporting service in a given namespace, you can choose between an enabled or disabled deployment. An enabled deployment signals the OpenShift Serverless Logic Operator to automatically intercept workflow deployments using the **preview** or **gitops** profile within the namespace and configure them to connect with the service.

For example, when the Data Index service is enabled, workflows are automatically configured to send status change events to it. Similarly, enabling the Job Service ensures that a job is created whenever a workflow requires a timeout. The OpenShift Serverless Logic Operator also configures the Job Service to send events to the Data Index service, facilitating seamless integration between the services.

The OpenShift Serverless Logic Operator does not just deploy supporting services, it also manages other necessary configurations to ensure successful workflow execution. All these configurations are handled automatically. You only need to provide the supporting services configuration in the **SonataFlowPlatform** CR.

NOTE

Deploying only one of the supporting services or using a disabled deployment are advanced use cases. In a standard installation, you must enable both services to ensure smooth workflow execution.

4.3.2. Supporting services deployment with the SonataFlowPlatform CR

To deploy supporting services, configure the **dataIndex** and **jobService** subfields within the **spec.services** section of the **SonataFlowPlatform** custom resource (CR). This configuration instructs the OpenShift Serverless Logic Operator to deploy each service when the **SonataFlowPlatform** CR is applied.

Each configuration of a service is handled independently, allowing you to customize these settings alongside other configurations in the **SonataFlowPlatform** CR.

See the following scaffold example configuration for deploying supporting services:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  services:
    dataIndex: (1)
      enabled: true (2)
      # Specific configurations for the Data Index Service
      # might be included here
    jobService: (3)
      enabled: true (4)
      # Specific configurations for the Job Service
      # might be included here
```

1. Data Index service configuration field.
2. Setting **enabled: true** deploys the Data Index service. If set to **false** or omitted, the deployment will be disabled. The default value is **false**.
3. Job Service configuration field.
4. Setting **enabled: true** deploys the Job Service. If set to **false** or omitted, the deployment will be disabled. The default value is **false**.

4.3.3. Supporting services scope

The **SonataFlowPlatform** custom resource (CR) enables the deployment of supporting services within a specific namespace. This means all automatically configured supporting services and workflow communications are restricted to the namespace of the deployed platform.

This feature is particularly useful when separate instances of supporting services are required for different sets of workflows. For example, you can deploy an application in isolation with its workflows and supporting services, ensuring they remain independent from other deployments.

4.3.4. Supporting services persistence configurations

The persistence configuration for supporting services in OpenShift Serverless Logic can be either ephemeral or PostgreSQL, depending on needs of your environment. Ephemeral persistence is ideal for development and testing, while PostgreSQL persistence is recommended for production environments.

4.3.4.1. Ephemeral persistence configuration

The ephemeral persistence uses an embedded PostgreSQL database that is dedicated to each service. The OpenShift Serverless Logic Operator recreates this database with every service restart, making it suitable only for development and testing purposes. You do not need any additional configuration other than the following **SonataFlowPlatform** CR:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  services:
    dataIndex:
      enabled: true
      # Specific configurations for the Data Index Service
      # might be included here
    jobService:
      enabled: true
      # Specific configurations for the Job Service
      # might be included here
```

4.3.4.2. PostgreSQL persistence configuration

For PostgreSQL persistence, you must set up a PostgreSQL server instance on your cluster. The administration of this instance remains independent of the OpenShift Serverless Logic Operator control. To connect a supporting service with the PostgreSQL server, you must configure the appropriate database connection parameters.

You can configure PostgreSQL persistence in the **SonataFlowPlatform** CR by using the following example:

Example of PostgreSQL persistence configuration

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  services:
    dataIndex:
      enabled: true
      persistence:
        postgresql:
          serviceRef:
            name: postgres-example (1)
```

```
        namespace: postgres-example-namespace (2)
        databaseName: example-database (3)
        databaseSchema: data-index-schema (4)
        port: 1234 (5)
      secretRef:
        name: postgres-secrets-example (6)
        userKey: POSTGRESQL_USER (7)
        passwordKey: POSTGRESQL_PASSWORD (8)
  jobService:
    enabled: true
    persistence:
      postgresql:
      # Specific database configuration for the Job Service
      # might be included here.
```

1. Name of the service to connect with the PostgreSQL database server.
2. Optional: Defines the namespace of the PostgreSQL Service. Defaults to the SonataFlowPlatform namespace.
3. Defines the name of the PostgreSQL database for storing supporting service data.
4. Optional: Specifies the schema for storing supporting service data. Default value is **SonataFlowPlatform** name, suffixed with **-data-index-service** or **-jobs-service**. For example, **sonataflow-platform-example-data-index-service**.
5. Optional: Port number to connect with the PostgreSQL Service. Default value is **5432**.
6. Defines the name of the secret containing the username and password for database access.
7. Defines the name of the key in the secret that contains the username to connect with the database.
8. Defines the name of the key in the secret that contains the password to connect with the database.

NOTE

You can configure each service's persistence independently by using the respective persistence field.

Create the secrets to access PostgreSQL by running the following command:

```
$ oc create secret generic <postgresql_secret_name> \
  --from-literal=POSTGRESQL_USER=<user> \
  --from-literal=POSTGRESQL_PASSWORD=<password> \
  -n <namespace>
```

4.3.4.3. Common PostgreSQL persistence configuration

The OpenShift Serverless Logic Operator automatically connects supporting services to the common PostgreSQL server configured in the **spec.persistence** field.

For rules, the following precedence is applicable:

- If you configure a specific persistence for a supporting service, for example, **services.dataIndex.persistence**, it uses that configuration.
- If you do not configure persistence for a service, the system uses the common persistence configuration from the current platform.

NOTE

When using a common PostgreSQL configuration, each service schema is automatically set as the **SonataFlowPlatform** name, suffixed with **-data-index-service** or **-jobs-service**, for example, **sonataflow-platform-example-data-index-service**.

4.3.5. Supporting services eventing system configurations

For a OpenShift Serverless Logic installation, the following types of events are generated:

- Outgoing and incoming events related to workflow business logic.
- Events sent from workflows to the Data Index and Job Service.
- Events sent from the Job Service to the Data Index Service.

The OpenShift Serverless Logic Operator leverages the Knative Eventing system to manage all event communication between these events and services, ensuring efficient and reliable event handling.

4.3.5.1. Platform-scoped eventing system configuration

To configure a platform-scoped eventing system, you can use the **spec.eventing.broker.ref** field in the **SonataFlowPlatform** CR to reference a Knative Eventing Broker. This configuration instructs the OpenShift Serverless Logic Operator to automatically link the supporting services to produce and consume events by using the specified broker.

A workflow deployed in the same namespace with the **preview** or **gitops** profile and without a custom eventing system configuration, automatically links to a specified broker.

IMPORTANT

In production environments, use a production-ready broker, such as the Knative Kafka Broker, for enhanced scalability and reliability.

The following example displays how to configure the **SonataFlowPlatform** CR for a platform-scoped eventing system:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  eventing:
    broker:
      ref:
```

```
    name: example-broker (1)
    namespace: example-broker-namespace (2)
    apiVersion: eventing.knative.dev/v1
    kind: Broker
```

1. Specifies the Knative Eventing Broker name.
2. Optional: Defines the namespace of the Knative Eventing Broker. If you do not specify a value, the parameter defaults to the **SonataFlowPlatform** namespace. Consider creating the Broker in the same namespace as **SonataFlowPlatform**.

4.3.5.2. Service-scoped eventing system configuration

A service-scoped eventing system configuration allows for fine-grained control over the eventing system, specifically for the Data Index or the Job Service.

> **NOTE**
>
> For a OpenShift Serverless Logic installation, consider using a platform-scoped eventing system configuration. The service-scoped configuration is intended for advanced use cases only.

4.3.5.3. Data Index eventing system configuration

To configure a service-scoped eventing system for the Data Index, you must use the **spec.services.dataIndex.source.ref** field in the **SonataFlowPlatform** CR to refer to a specific Knative Eventing Broker. This configuration instructs the OpenShift Serverless Logic Operator to automatically link the Data Index to consume SonataFlow system events from that Broker.

> **IMPORTANT**
>
> In production environments, use a production-ready broker, such as the Knative Kafka Broker, for enhanced scalability and reliability.

The following example displays the Data Index eventing system configuration:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
spec:
  services:
    dataIndex:
      source:
        ref:
          name: data-index-source-example-broker (1)
          namespace: data-index-source-example-broker-namespace (2)
          apiVersion: eventing.knative.dev/v1
          kind: Broker
```

1. Specifies the Knative Eventing Broker from which the Data Index consumes events.

2. Optional: Defines the namespace of the Knative Eventing Broker. If you do not specify a value, the parameter defaults to the **SonataFlowPlatform** namespace. Consider creating the broker in the

4.3.5.4. Job Service eventing system configuration

To configure a service-scoped eventing system for the Job Service, you must use the **spec.services.jobService.source.ref** and **spec.services.jobService.sink.ref** fields in the **SonataFlowPlatform** CR. These fields instruct the OpenShift Serverless Logic Operator to automatically link the Job Service to consume and produce SonataFlow system events, based on the provided configuration.

> **IMPORTANT**
>
> In production environments, use a production-ready broker, such as the Knative Kafka Broker, for enhanced scalability and reliability.

The following example displays the Job Service eventing system configuration:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
spec:
  services:
    jobService:
      source:
        ref:
          name: jobs-service-source-example-broker (1)
          namespace: jobs-service-source-example-broker-namespace (2)
          apiVersion: eventing.knative.dev/v1
          kind: Broker
      sink:
        ref:
          name: jobs-service-sink-example-broker (3)
          namespace: jobs-service-sink-example-broker-namespace (4)
          apiVersion: eventing.knative.dev/v1
          kind: Broker
```

1. Specifies the Knative Eventing Broker from which the Job Service consumes events.
2. Optional: Defines the namespace of the Knative Eventing Broker. If you do not specify a value, the parameter defaults to the **SonataFlowPlatform** namespace. Consider creating the Broker in the same namespace as **SonataFlowPlatform**.
3. Specifies the Knative Eventing Broker on which the Job Service produces events.
4. Optional: Defines the namespace of the Knative Eventing Broker. If you do not specify a value, the parameter defaults to the **SonataFlowPlatform** namespace. Consider creating the Broker in the same namespace as **SonataFlowPlatform**.

4.3.5.5. Cluster-scoped eventing system configuration for supporting services

When you deploy cluster-scoped supporting services, the supporting services automatically link to the Broker specified in the **SonataFlowPlatform** CR, which is referenced by the **SonataFlowClusterPlatform** CR.

4.3.5.6. Eventing system configuration precedence rules for supporting services

The OpenShift Serverless Logic Operator follows a defined order of precedence to configure the eventing system for a supporting service.

Eventing system configuration precedence rules are as follows:

1. If the supporting service has its own eventing system configuration, using either the Data Index eventing system or the Job Service eventing system configuration, then supporting service configuration takes precedence.
2. If the **SonataFlowPlatform** CR enclosing the supporting service is configured with a platform-scoped eventing system, that configuration takes precedence.
3. If the current cluster is configured with a cluster-scoped eventing system, that configuration takes precedence.
4. f none of the previous configurations exist, the supporting service delivers events by direct HTTP calls.

4.3.5.7. Eventing system linking configuration

The OpenShift Serverless Logic Operator automatically creates Knative Eventing, **SinkBindings**, and triggers to link supporting services with the eventing system. These objects enable the production and consumption of events by the supporting services.

The following example displays the Knative Native eventing objects created for the **SonataFlowPlatform** CR:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  eventing:
    broker:
      ref:
        name: example-broker (1)
        apiVersion: eventing.knative.dev/v1
        kind: Broker
  services:
    dataIndex: (2)
      enabled: true
    jobService: (3)
      enabled: true
```

1. Used by the Data Index, Job Service, and workflows, unless overridden.
2. Data Index ephemeral deployment, configures the Data Index service.

3 Job Service ephemeral deployment, configures the Job Service.

The following example displays how to configure a Knative Kafka Broker for use with the **SonataFlowPlatform** CR:

Example of Knative Kafka Broker example used by the SonataFlowPlatform CR

```
apiVersion: eventing.knative.dev/v1
kind: Broker
metadata:
  annotations:
    eventing.knative.dev/broker.class: Kafka 1
  name: example-broker
  namespace: example-namespace
spec:
  config:
    apiVersion: v1
    kind: ConfigMap
    name: kafka-broker-config
    namespace: knative-eventing
```

1 Use the Kafka class to create a Kafka Knative Broker.

The following command displays the list of triggers set up for the Data Index and Job Service events, showing which services are subscribed to the events:

```
$ oc get triggers -n example-namespace
```

Example output

```
NAME                                                    BROKER          SINK                                                    AGE
CONDITIONS   READY   REASON
data-index-jobs-fbf285df-c0a4-4545-b77a-c232ec2890e2        example-broker   service:sonataflow-
platform-example-data-index-service     106s  7 OK / 7    True    -
data-index-process-definition-e48b4e4bf73e22b90ecf7e093ff6b1eaf   example-broker
service:sonataflow-platform-example-data-index-service     106s  7 OK / 7    True    -
data-index-process-error-fbf285df-c0a4-4545-b77a-c232ec2890e2   example-broker
service:sonataflow-platform-example-data-index-service     106s  7 OK / 7    True    -
data-index-process-instance-mul35f055c67a626f51bb8d2752606a6b54   example-broker
service:sonataflow-platform-example-data-index-service     106s  7 OK / 7    True    -
data-index-process-node-fbf285df-c0a4-4545-b77a-c232ec2890e2      example-broker
service:sonataflow-platform-example-data-index-service     106s  7 OK / 7    True    -
data-index-process-state-fbf285df-c0a4-4545-b77a-c232ec2890e2     example-broker
service:sonataflow-platform-example-data-index-service     106s  7 OK / 7    True    -
data-index-process-variable-ac727d6051750888dedb72f697737c0dfbf   example-broker
service:sonataflow-platform-example-data-index-service     106s  7 OK / 7    True    -
jobs-service-create-job-fbf285df-c0a4-4545-b77a-c232ec2890e2    example-broker
service:sonataflow-platform-example-jobs-service         106s  7 OK / 7    True    -
jobs-service-delete-job-fbf285df-c0a4-4545-b77a-c232ec2890e2    example-broker
service:sonataflow-platform-example-jobs-service         106s  7 OK / 7    True    -
```

To see the **SinkBinding** resource for the Job Service, use the following command:

```
$ oc get sources -n example-namespace
```

Example output

```
NAME                                TYPE          RESOURCE                    SINK                READY
sonataflow-platform-example-jobs-service-sb   SinkBinding   sinkbindings.sources.knative.dev
broker:example-broker   True
```

4.3.6. Advanced supporting services configurations

In scenarios where you must apply advanced configurations for supporting services, use the **podTemplate** field in the **SonataFlowPlatform** custom resource (CR). This field allows you to customize the service pod deployment by specifying configurations like the number of replicas, environment variables, container images, and initialization options.

You can configure advanced settings for the service by using the following example:

Advanced configurations example for the Data Index service

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  services:
    # This can be either 'dataIndex' or 'jobService'
    dataIndex:
      enabled: true
      podTemplate:
        replicas: 2 (1)
        container: (2)
          env: (3)
            - name: <any_advanced_config_property>
              value: <any_value>
          image: (4)
        initContainers: (5)
```

NOTE

You can set the 'services' field to either 'dataIndex' or 'jobService' depending on your requirement. The rest of the configuration remains the same.

1. Defines the number of replicas. Default value is **1**. In the case of **jobService**, this value is always overridden to **1** because it operates as a singleton service.
2. Holds specific configurations for the container running the service.
3. Allows you to fine-tune service properties by specifying environment variables.
4. Configures the container image for the service, useful if you need to update or customize the image.

5 Configures init containers for the pod, useful for setting up prerequisites before the main container starts.

NOTE

The **podTemplate** field provides flexibility for tailoring the deployment of each supporting service. It follows the standard **PodSpec** API, meaning the same API validation rules apply to these fields.

4.3.7. Cluster scoped supporting services

You can define a cluster-wide set of supporting services that can be consumed by workflows across different namespaces, by using the **SonataFlowClusterPlatform** custom resource (CR). By referencing an existing namespace-specific **SonataFlowPlatform** CR, you can extend the use of these services cluster-wide.

You can use the following example of a basic configuration that enables workflows deployed in any namespace to utilize supporting services deployed in a specific namespace, such as **example-namespace**:

Example of a SonataFlowClusterPlatform CR

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowClusterPlatform
metadata:
  name: cluster-platform
spec:
  platformRef:
    name: sonataflow-platform-example (1)
    namespace: example-namespace (2)
```

1 Specifies the name of the already installed **SonataFlowPlatform** CR that manages the supporting services.

2 Specifies the namespace of the **SonataFlowPlatform** CR that manages the supporting services.

NOTE

You can override these cluster-wide services within any namespace by configuring that namespace in **SonataFlowPlatform.spec.services**.

CHAPTER 5. MANAGING WORKFLOW PERSISTENCE

You can configure a **SonataFlow** instance to use persistence and store workflow context in a relational database.

By design, Kubernetes pods are stateless. This behavior can pose challenges for workloads that need to maintain the application state across pod restarts. In the case of OpenShift Serverless Logic, the workflow context is lost when the pod restarts by default.

To ensure workflow recovery in such scenarios, you must configure workflow runtime persistence. Use the **SonataFlowPlatform** custom resource (CR) or the **SonataFlow** CR to provide this configuration. The scope of the configuration varies depending on which resource you use.

5.1. CONFIGURING PERSISTENCE USING THE SONATAFLOWPLATFORM CR

The **SonataFlowPlatform** custom resource (CR) enables persistence configuration at the namespace level. This approach applies the persistence settings automatically to all workflows deployed in the namespace. It simplifies resource configuration, especially when multiple workflows in the namespace belong to the same application. While this configuration is applied by default, individual workflows in the namespace can override it using the **SonataFlow** CR.

The OpenShift Serverless Logic Operator also uses this configuration to set up persistence for supporting services.

NOTE

The persistence configurations are applied only at the time of workflow deployment. Changes to the **SonataFlowPlatform** CR do not affect workflows that are already deployed.

Procedure

1. Define the **SonataFlowPlatform** CR.
2. Specify the persistence settings in the **persistence** field under the **SonataFlowPlatform** CR spec.

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  persistence:
    postgresql:
      serviceRef:
        name: postgres-example (1)
        namespace: postgres-example-namespace (2)
        databaseName: example-database (3)
        port: 1234 (4)
      secretRef:
```

```
name: postgres-secrets-example (5)
userKey: POSTGRESQL_USER (6)
passwordKey: POSTGRESQL_PASSWORD (7)
```

1. Name of the Kubernetes Service connecting to the PostgreSQL database.
2. Optional: Namespace of the PostgreSQL Service. Defaults to the namespace of the **SonataFlowPlatform**.
3. Name of the PostgreSQL database for storing workflow data.
4. Optional: Port number to connect to the PostgreSQL service. Defaults to **5432**.
5. Name of the Kubernetes Secret containing database credentials.
6. Key in the **Secret** object that contains the database username.
7. Key in the **Secret** object that contains the database password.

3. View the generated environment variables for the workflow.
   The following example shows the generated environment variables for a workflow named **example-workflow** deployed with the earlier **SonataFlowPlatform** configuration. These configurations specifically relate to persistence and are managed by the OpenShift Serverless Logic Operator. You cannot modify these settings once you have applied them.

> **NOTE**
>
> When you use the **SonataFlowPlatform** persistence, every workflow is configured to use a PostgreSQL schema name equal to the workflow name.

```
env:
  - name: QUARKUS_DATASOURCE_USERNAME
    valueFrom:
      secretKeyRef:
        name: postgres-secrets-example
        key: POSTGRESQL_USER
  - name: QUARKUS_DATASOURCE_PASSWORD
    valueFrom:
      secretKeyRef:
        name: postgres-secrets-example
        key: POSTGRESQL_PASSWORD
  - name: QUARKUS_DATASOURCE_DB_KIND
    value: postgresql
  - name: QUARKUS_DATASOURCE_JDBC_URL
    value: >-
      jdbc:postgresql://postgres-example.postgres-example-namespace:1234/example-database?
currentSchema=example-workflow
  - name: KOGITO_PERSISTENCE_TYPE
    value: jdbc
```

When this persistence configuration is in place, the OpenShift Serverless Logic Operator configures every workflow deployed in this namespace using the **preview** or **gitops** profile, to connect with the PostgreSQL database by injecting relevant JDBC connection parameters as environment variables.

NOTE

PostgreSQL is currently the only supported database for persistence.

For **SonataFlow** CR deployments using the **preview** profile, the OpenShift Serverless Logic build system automatically includes specific Quarkus extensions required for enabling persistence. This ensures compatibility with persistence mechanisms, streamlining the workflow deployment process.

5.2. CONFIGURING PERSISTENCE USING THE SONATAFLOW CR

The **SonataFlow** custom resource (CR) enables workflow-specific persistence configuration. You can use this configuration independently, even if **SonataFlowPlatform** persistence is already set up in the current namespace.

Procedure

- Configure persistence by using the **persistence** field in the **SonataFlow** CR specification as shown in the following example:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: example-workflow
  annotations:
    sonataflow.org/description: Example Workflow
    sonataflow.org/version: 0.0.1
spec:
  persistence:
    postgresql:
      serviceRef:
        name: postgres-example (1)
        namespace: postgres-example-namespace (2)
        databaseName: example-database (3)
        databaseSchema: example-schema (4)
        port: 1234 (5)
      secretRef:
        name: postgres-secrets-example (6)
        userKey: POSTGRESQL_USER (7)
        passwordKey: POSTGRESQL_PASSWORD (8)
  flow:
```

1. Name of the Kubernetes Service that connects to the PostgreSQL database server.
2. Optional: Namespace containing the PostgreSQL Service. Defaults to the workflow namespace.
3. Name of the PostgreSQL database where workflow data is stored.
4. Optional: Name of the database schema for workflow data. Defaults to the workflow name.
5. Optional: Port to connect to the PostgreSQL Service. Defaults to **5432**.
6. Name of the Kubernetes Secret containing database credentials.

7. Key in the **Secret** object containing the database username.
8. Key in the **Secret** object containing the database password.

This configuration informs the OpenShift Serverless Logic Operator that the workflow must connect to the specified PostgreSQL database server when deployed. The OpenShift Serverless Logic Operator adds the relevant JDBC connection parameters as environment variables to the workflow container.

> **NOTE**
>
> PostgreSQL is currently the only supported database for persistence.

For **SonataFlow** CR deployments using the **preview** profile, the OpenShift Serverless Logic build system includes the required Quarkus extensions to enable persistence automatically.

5.3. PERSISTENCE CONFIGURATION PRECEDENCE RULES

You can use **SonataFlow** custom resource (CR) persistence independently or alongside **SonataFlowPlatform** CR persistence. If a **SonataFlowPlatform** CR persistence configuration exists in the current namespace, the following rules determine which persistence configuration applies:

1. If the **SonataFlow** CR includes a persistence configuration, that configuration takes precedence and applies to the workflow.
2. If the **SonataFlow** CR does not include a persistence configuration and the **spec.persistence** field is absent, the OpenShift Serverless Logic Operator uses the persistence configuration from the current **SonataFlowPlatform** if any.
3. To disable persistence for the workflow, explicitly set **spec.persistence: {}** in the **SonataFlow** CR. This configuration ensures the workflow does not inherit persistence settings from the **SonataFlowPlatform** CR.

5.4. PROFILE SPECIFIC PERSISTENCE REQUIREMENTS

The persistence configurations provided for both **SonataFlowPlatform** custom resource (CR) and **SonataFlow** CR apply equally to the **preview** and **gitops** profiles. However, you must avoid using these configurations with the **dev** profile, as this profile ignores them entirely.

The primary difference between the **preview** and **gitops** profiles lies in the build process.

When using the **gitops** profile, ensure that the following Quarkus extensions are included in the workflow image during the build process.

groupId	artifactId	version
io.quarkus	quarkus-agroal	3.8.6.redhat-00004
io.quarkus	quarkus-jdbc-postgresql	3.8.6.redhat-00004

groupId	artifactId	version
org.kie	**kie-addons-quarkus-persistence-jdbc**	**9.102.0.redhat-00005**

If you are using the **registry.redhat.io/openshift-serverless-1/logic-swf-builder-rhel8:1.35.0** to generate your images, you can pass the following build argument to include these extensions:

```
$ QUARKUS_EXTENSIONS=io.quarkus:quarkus-agroal:3.8.6.redhat-00004,io.quarkus:quarkus-jdbc-postgresql:3.8.6.redhat-00004,org.kie:kie-addons-quarkus-persistence-jdbc:9.102.0.redhat-00005
```

5.5. DATABASE SCHEMA INITIALIZATION

When you are using **SonataFlow** with PostgreSQL persistence, you can initialize the database schema either by enabling Flyway or by manually applying database schema updates using Data Definition Language (DDL) scripts.

Flyway is managed by the **kie-addons-quarkus-flyway** runtime module and it is disabled by default. To enable Flyway, you must configure it using one of the following methods:

5.5.1. Flyway configuration in the workflow ConfigMap

To enable Flyway in the workflow **ConfigMap**, you can add the following property:

Example of enabling Flyway in the workflow ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app: example-workflow
  name: example-workflow-props
data:
  application.properties: |
    kie.flyway.enabled = true
```

5.5.2. Flyway configuration using environment variables in the workflow container

You can enable Flyway by adding an environment variable to the **spec.podTemplate.container** field in the **SonataFlow** CR by using the following example:

Example of enabling Flyway by using the workflow container environment variable

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: example-workflow
  annotations:
    sonataflow.org/description: Example Workflow
    sonataflow.org/version: 0.0.1
spec:
```

```
podTemplate:
  container:
    env:
      - name: KIE_FLYWAY_ENABLED
        value: 'true'
flow: ...
```

5.5.3. Flyway configuration using SonataFlowPlatform properties

To apply a common Flyway configuration to all workflows within a namespace, you can add the property to the **spec.properties.flow** field of the **SonataFlowPlatform** CR shown in the following example:

> **NOTE**
>
> This configuration is applied during workflow deployment. Ensure the Flyway property is set before deploying workflows.

Example of enabling Flyway by using the SonataFlowPlatform properties

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform
spec:
  properties:
    flow:
      - name: kie.flyway.enabled
        value: true
```

5.5.4. Initializing a manual database using DDL scripts

If you prefer manual initialization, you must disable Flyway by ensuring the **kie.flyway.enabled** property is either not configured or explicitly set to **false**.

- By default, each workflow uses a schema name equal to the workflow name. Ensure that you manually apply the schema initialization for each workflow.
- If you are using the **SonataFlow** custom resource (CR) persistence configuration, you can specify a custom schema name.

Procedure

1. Download the DDL scripts from the kogito-ddl-9.102.0.redhat-00005-db-scripts.zip location.
2. Extract the files.
3. Run the **.sql** files located in the root directory on the target PostgreSQL database. Ensure that the files are executed in the order of their version numbers.
   For example:
   - **V1.35.0__create_runtime_PostgreSQL.sql**
   - **V10.0.0__add_business_key_PostgreSQL.sql**

- **V10.0.1__alter_correlation_PostgreSQL.sql**

NOTE

The file version numbers are not associated with the OpenShift Serverless Logic Operator versioning.

CHAPTER 6. WORKFLOW EVENTING SYSTEM

You can set up the eventing system for a **SonataFlow** workflow.

In a OpenShift Serverless Logic installation, the following types of events are generated:

- Outgoing and incoming events related to workflow business logic.
- Events sent from workflows to the Data Index and Job Service.
- Events sent from the Job Service to the Data Index Service.

The OpenShift Serverless Logic Operator leverages the Knative Eventing system to manage all event communication between these services, ensuring efficient and reliable event handling.

6.1. PLATFORM-SCOPED EVENTING SYSTEM CONFIGURATION

To configure a platform-scoped eventing system, you can use the **spec.eventing.broker.ref** field in the **SonataFlowPlatform** custom resource (CR) to reference a Knative Eventing broker.

This configuration instructs the OpenShift Serverless Logic Operator to automatically link every workflow deployed in the specified namespace, using the **preview** or **gitops** profile, to produce and consume events through the defined broker.

The supporting services deployed in the namespace without a custom eventing configuration are also linked to this broker.

> **NOTE**
>
> In production environments, use a production-ready broker, such as the Knative Kafka Broker, for enhanced scalability and reliability.

The following example displays how to configure the **SonataFlowPlatform** CR for a platform-scoped eventing system:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: <example-namespace>
spec:
  eventing:
    broker:
      ref:
        name: example-broker (1)
        namespace: <example-broker-namespace> (2)
        apiVersion: eventing.knative.dev/v1
        kind: Broker
```

1. Specifies the Knative Eventing Broker name.
2. Optional: Specifies the namespace of the Knative Eventing Broker. If you do not sepcify the value, then the parameter defaults to the namespace of the **SonataFlowPlatform** CR. Consider creating the broker in the same namespace as **SonataFlowPlatform**.

6.2. WORKFLOW-SCOPED EVENTING SYSTEM CONFIGURATION

A workflow-scoped eventing system configuration allows for detailed customization of the events produced and consumed by a specific workflow. You can use the **spec.sink.ref** and **spec.sources[]** fields in the **SonataFlow** CR to configure outgoing and incoming events.

6.2.1. Outgoing eventing system configuration

To configure outgoing events, you can use the **spec.sink.ref** field in the **SonataFlow** CR. This configuration ensures the workflow produces events using the specified Knative Eventing Broker, including both system events and workflow business events.

The following example displays how to configure the **SonataFlow** CR for a workflow-scoped outgoing eventing system:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: example-workflow
  namespace: example-workflow-namespace
  annotations:
    sonataflow.org/description: Example Workflow
    sonataflow.org/version: 0.0.1
    sonataflow.org/profile: preview
spec:
  sink:
    ref:
      name: outgoing-example-broker (1)
      namespace: outgoing-example-broker-namespace (2)
      apiVersion: eventing.knative.dev/v1
      kind: Broker
  flow: (3)
    start: ExampleStartState
    events: (4)
      - name: outEvent1 (5)
        source: ''
        kind: produced
        type: out-event-type1 (6)
    ...
```

1. Name of the Knative Eventing Broker to use for all the events produced by the workflow, including the SonataFlow system events.
2. Defines the namespace of the Knative Eventing Broker. If you do not specify a value, the parameter defaults to the **SonataFlow** namespace. Consider creating the broker in the same namespace as **SonataFlow**.
3. Flow definition field in the **SonataFlow** CR.
4. Events definition field in the **SonataFlow** CR.
5. Example of an outgoing event **outEvent1** definition.
6. Event type for the **outEvent1** outgoing event.

6.2.2. Incoming eventing system configuration

To configure incoming events, you can use the **spec.sources[]** field in the **SonataFlow** CR. You can add an entry for each event type requiring specific configuration. This setup allows workflows to consume events from different brokers based on event type.

If an incoming event type lacks a specific Broker configuration, the system applies eventing system configuration precedence rules.

The following example displays how to configure the **SonataFlow** CR for a workflow-scoped incoming eventing system:

> **NOTE**
>
> The link between a **spec.sources[]** entry and the workflow event, is by using the event type.

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: example-workflow
  namespace: example-workflow-namespace
  annotations:
    sonataflow.org/description: Example Workflow
    sonataflow.org/version: 0.0.1
    sonataflow.org/profile: preview
spec:
  sources:
    - eventType: in-event-type1 (1)
      ref:
        name: incoming-example-broker1 (2)
        namespace: incoming-example-broker1-namespace (3)
        apiVersion: eventing.knative.dev/v1
        kind: Broker
    - eventType: in-event-type2 (4)
      ref:
        name: incoming-example-broker2 (5)
        namespace: incoming-example-broker2-namespace (6)
        apiVersion: eventing.knative.dev/v1
        kind: Broker
  flow: (7)
    start: ExampleStartState
    events: (8)
      - name: inEvent1 (9)
        source: ''
        kind: consumed
        type: in-event-type1 (10)
      - name: inEvent2 (11)
        source: ''
        kind: consumed
        type: in-event-type2 (12)
    ...
```

1. Configure the workflow to consume events of type **in-event-type1** using the specified Knative Eventing Broker.
2. Name of the Knative Eventing Broker to use for the consumption of the events of type **in-event-type1** sent to this workflow.
3. Optional: If you do not specify the value, the parameter defaults to the **SonataFlow** namespace. Consider creating the broker in the same namespace as the **SonataFlow** workflow.
4. Configure the workflow to consume events of type **in-event-type2** using the specified Knative Eventing Broker.
5. Name of the Knative Eventing Broker to use for the consumption of the events of type **in-event-type2** sent to this workflow.
6. Optional: If you do not specify the value, the parameter defaults to the **SonataFlow** namespace. Consider creating the broker in the same namespace as the **SonataFlow** workflow.
7. Flow definition field in the **SonataFlow** CR.
8. Events definition field in the **SonataFlow** CR.
9. Example of an incoming event **inEvent1** definition.
10. Event type for the incoming event **inEvent1**. The link of the workflow event, with the corresponding **spec.sources[]** entry, is by using the event type name **in-event-type1**.
11. Example of an incoming event **inEvent2** definition.
12. Event type for the incoming event **inEvent2**. The link of the workflow event, with the corresponding spec.sources[] entry, is created by using the event type name in-event-type2.

6.3. CLUSTER-SCOPED EVENTING SYSTEM CONFIGURATION

In a **SonataFlowClusterPlatform** setup, workflows are automatically linked to the Broker specified in the associated **SonataFlowPlatform** CR. This linkage follows the Eventing System configuration precedence rules.

To ensure proper integration, you can configure **Broker** in the **SonataFlowPlatform** CR referenced by the **SonataFlowClusterPlatform** CR.

The following example displays how to configure the **SonataFlowClusterPlatform** CR and its reference to the **SonataFlowPlatform** CR:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: global-platform
  namespace: global-namespace
spec:
  eventing:
    broker:
      ref:
        name: global-broker
```

```
        namespace: global-namespace
        apiVersion: eventing.knative.dev/v1
        kind: Broker
---
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowClusterPlatform
metadata:
  name: cluster-platform-example
spec:
  platformRef:
    name: global-platform
    namespace: global-namespace
    ...
```

NOTE

The **SonataFlowClusterPlatform** CR can refer to any **SonataFlowPlatform** CR that has already been deployed.

6.4. EVENTING SYSTEM CONFIGURATION PRECEDENCE RULES

The OpenShift Serverless Logic Operator follows a defined order of precedence to determine the eventing system configuration for a workflow.

Eventing system configuration precedence rules are as follows:

1. If the workflow has a defined eventing system by using either workflow-scoped outgoing or incoming eventing system configurations, the choice of configuration takes priority over the other configuration and applies to the workflow.
2. If the **SonataFlowPlatform** CR enclosing the workflow has a platform-scoped eventing system configured, this configuration is then applied to the next step.
3. If the current cluster is configured with a cluster-scoped eventing system, it is applied if no workflow-scoped or platform-scoped configuration exists.
4. Review the following information, if none of the above configurations are defined:
   - The workflow uses direct HTTP calls to deliver SonataFlow system events to supporting services.
   - The workflow consumes incoming events by HTTP POST calls at the workflow service root path /.
   - No eventing system is configured to produce workflow business events. Any attempt to produce such events might result in a failure.

6.5. LINKING WORKFLOWS TO THE EVENTING SYSTEM

The OpenShift Serverless Logic Operator links workflows with the eventing system using Knative Eventing, SinkBindings, and triggers. These objects are created automatically by the OpenShift Serverless Logic Operator and simplify the production and consumption of workflow events.

The following example shows the Knative Eventing objects created for an **example-workflow** workflow configured with a platform-scoped eventing system:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowPlatform
metadata:
  name: sonataflow-platform-example
  namespace: example-namespace
spec:
  eventing:
    broker:
      ref:
        name: example-broker (1)
        apiVersion: eventing.knative.dev/v1
        kind: Broker
  services:
    dataIndex: (2)
      enabled: true
    jobService: (3)
      enabled: true
  ...
```

1. The **example-broker** object is used by the Data Index, Jobs Service, and the **example-workflow** workflow.
2. The Data Index is deployed with ephemeral configurations.
3. The Jobs Service is deployed with ephemeral configurations.

The **example-broker** object is a Kafka class Broker, and its configuration is defined in the **kafka-broker-config** config map.

The following example displays how to configure a Kafka Knative Broker for use with the SonataFlowPlatform:

```
apiVersion: eventing.knative.dev/v1
kind: Broker
metadata:
  annotations:
    eventing.knative.dev/broker.class: Kafka (1)
  name: example-broker
  namespace: example-namespace
spec:
  config:
    apiVersion: v1
    kind: ConfigMap
    name: kafka-broker-config
    namespace: knative-eventing
```

1. The Kafka class is used to create the **example-broker** object.

The following example displays how the **example-workflow** is automatically linked to the **example-broker** in the **example-namespace** for event production and consumption:

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
```

```
metadata:
  name: example-workflow
  namespace: example-namespace
  annotations:
    sonataflow.org/description: Example Workflow
    sonataflow.org/version: 0.0.1
    sonataflow.org/profile: preview
spec:
  flow:
    start: ExampleStartState
    events:
      - name: outEvent1
        source: ''
        kind: produced
        type: out-event-type1 (1)
      - name: inEvent1
        source: ''
        kind: consumed
        type: in-event-type1 (2)
      - name: inEvent2
        source: ''
        kind: consumed
        type: in-event-type2 (3)
    states:
      - name: ExampleStartState
    ...
```

1. The **example-workflow** outgoing events are produced by using the **SinkBinding** named **example-workflow-sb**.
2. Events of type **in-event-type1** are consumed by using the **example-workflow-inevent1-b40c067c-595b-4913-81a4-c8efa980bc11** trigger.
3. Events of type **in-event-type2** are consumed by using the **example-workflow-inevent2-b40c067c-595b-4913-81a4-c8efa980bc11** trigger.

You can list the automatically created **SinkBinding** named **example-workflow-sb** by using the following command:

```
$ oc get sinkbindings -n example-namespace
```

Example output

```
NAME                 TYPE         RESOURCE                        SINK                 READY
example-workflow-sb  SinkBinding  sinkbindings.sources.knative.dev  broker:example-broker
True
```

You can use the following command to list the automatically created triggers for event consumption:

```
$ oc get triggers -n <example-namespace>
```

Example output

```

```

```
NAME                                                    BROKER          SINK
AGE   CONDITIONS   READY   REASON
example-workflow-inevent1-b40c067c-595b-4913-81a4-c8efa980bc11    example-broker
service:example-workflow                                 16m   7 OK / 7     True
example-workflow-inevent2-b40c067c-595b-4913-81a4-c8efa980bc11    example-broker
service:example-workflow                                 16m   7 OK / 7     True
```

CHAPTER 7. CONFIGURING CUSTOM MAVEN MIRRORS

OpenShift Serverless Logic uses Maven Central by default to resolve Maven artifacts during workflow builds. The provided builder and development images include all required Java libraries to run workflows, but in certain scenarios, such as when you add a custom Quarkus extension, you must download the additional dependencies from Maven Central.

In environments with restricted or firewalled network access, direct access to Maven Central might not be available. In such cases, you can configure the workflow containers to use a custom Maven mirror, such as an internal company registry or repository manager.

You can configure a custom Maven mirror at different levels as follows:

- Per workflow build by updating the **SonataFlowBuild** custom resource.
- At the platform level by updating the **SonataFlowPlatform** custom resource.
- For development mode deployments by editing the **SonataFlow** custom resource.
- When building custom images externally with the builder image

7.1. ADDING A MAVEN MIRROR WHEN BUILDING WORKFLOWS

You can configure a Maven mirror by setting the **MAVEN_MIRROR_URL** environment variable in the **SonataFlowBuild** or **SonataFlowPlatform** custom resources (CR).

NOTE

The recommended approach is to update the **SonataFlowPlatform** CR. This ensures the mirror configuration is propagated automatically to all workflow builds within the platform scope.

Prerequisites

- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have created your OpenShift Serverless Logic project.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to a custom Maven mirror or internal repository.

Procedure

1. Edit the **SonataFlowPlatform** CR to configure a Maven mirror for all workflow builds in a namespace, as shown in the following example:

   Example of Maven mirror configuration in a SonataFlowPlatform CR

   ```
   apiVersion: sonataflow.org/v1alpha08
   kind: SonataFlowPlatform
   metadata:
     name: my-platform
   ```

```
spec:
  build:
    template:
      envs:
        - name: MAVEN_MIRROR_URL
          value: http://my.company.registry.local
```

This configuration applies to all workflow builds in the same namespace that use the **preview** profile. When a workflow builder instance runs, it updates the internal Maven settings file to use the specified mirror as the default for external locations such as Maven Central.

2. Optional: If you need a specific configuration for a single workflow build, create the **SonataFlowBuild** CR before creating the corresponding **SonataFlow** CR. The **SonataFlowBuild** and **SonataFlow** CRs must have the same name.

Example of Maven mirror configuration in a SonataFlowBuild CR

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlowBuild
metadata:
  name: my-workflow (1)
  annotations:
    sonataflow.org/restartBuild: "true" (2)
spec:
  # suppressed for brevity
  envs:
    - name: MAVEN_MIRROR_URL (3)
      value: http://my.company.registry.local
```

1. The **SonataFlowBuild** CR must have the same name as the corresponding **SonataFlow** CR.
2. The **sonataflow.org/restartBuild: "true"** annotation forces the existing build to restart with the new configuration.
3. The **MAVEN_MIRROR_URL** environment variable specifies the custom Maven mirror.

NOTE

You can use the **SonataFlowBuild** CR configuration only when you require workflow-specific behavior, for example, debugging. For general use, configure the **SonataFlowPlatform** CR instead.

7.2. ADDING A MAVEN MIRROR WHEN DEPLOYING IN DEVELOPMENT MODE

You can configure a Maven mirror for workflows that run in **dev** mode by adding the **MAVEN_MIRROR_URL** environment variable to the **SonataFlow** custom resource (CR).

Prerequisites

- You have created your OpenShift Serverless Logic project.

- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have a workflow deployed in **dev** profile.
- You have access to a custom Maven mirror or internal repository.

Procedure

- Edit the **SonataFlow** CR to include the Maven mirror configuration as shown in the following example:

Example of Maven mirror configuration on SonataFlow CR

```
apiVersion: sonataflow.org/v1alpha08
kind: SonataFlow
metadata:
  name: greeting
  annotations:
    sonataflow.org/description: Greeting example on k8s!
    sonataflow.org/version: 0.0.1
    sonataflow.org/profile: dev
spec:
  podTemplate:
    container:
      env:
        - name: MAVEN_MIRROR_URL (1)
          value: http://my.company.registry.local
  flow: #suppressed for brevity
```

(1) The **MAVEN_MIRROR_URL** variable specifies the custom Maven mirror.

> **NOTE**
>
> Only workflows deployed with the **dev** profile can use Maven mirrors. Other deployment models run compiled code only, so they do not need to connect to a Maven registry.

7.3. CONFIGURING A MAVEN MIRROR ON A CUSTOM IMAGE

You can configure a Maven mirror for workflows that run in **dev** mode by adding the **MAVEN_MIRROR_URL** environment variable to the **SonataFlow** custom resource (CR).

Prerequisites

- You have created your OpenShift Serverless Logic project.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to a dockerfile or container build context that uses the SonataFlow Builder image.
- You have access to a custom Maven mirror or internal repository.

Procedure

1. Set the Maven mirror as an environment variable in the Dockerfile as shown in the following example:

   Example of custom container file with Maven mirror set as an environment variable

   ```
   FROM docker.io/apache/incubator-kie-sonataflow-builder:main AS builder

   # Content suppressed for brevity

   # The Maven mirror URL set as an env var during the build process
   ENV MAVEN_MIRROR_URL=http://my.company.registry.local
   ```

   The **ENV** directive ensures that all builds with this Dockerfile automatically use the specified Maven mirror.

2. Set the Maven mirror as a build-time argument in the Dockerfile as shown in the following example:

   Example of custom container file with Maven mirror set as an argument

   ```
   FROM docker.io/apache/incubator-kie-sonataflow-builder:main AS builder

   # Content suppressed for brevity

   # The Maven mirror URL passed as a build argument during the build process
   ARG MAVEN_MIRROR_URL
   ```

   The **ARG** directive allows you to pass the Maven mirror value dynamically at build time.

CHAPTER 8. MANAGING UPGRADES

8.1. UPGRADING OPENSHIFT SERVERLESS LOGIC OPERATOR FROM VERSION 1.34.0 TO 1.35.0

This section provides step-by-step instructions to upgrade the OpenShift Serverless Logic Operator from version 1.34.0 to 1.35.0. The upgrade process involves preparing the existing workflows and services, updating the Operator, and restoring the workflows after the upgrade.

NOTE

Different workflow profiles require different upgrade steps. Carefully follow the instructions for each profile.

8.1.1. Preparing for the upgrade

Before starting the upgrade process, you need to prepare your OpenShift Serverless Logic environment. This section outlines the necessary steps to ensure a upgrade from version 1.34.0 to 1.35.0.

The preparation process includes:

- Deleting or scaling workflows based on their profiles.
- Backing up all necessary databases and resources.
- Ensuring you have a record of all custom configurations.
- Running required database migration scripts for workflows using persistence.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to the **OpenShift Management Console** for Operator upgrades.
- You have installed the OpenShift CLI (**oc**).

8.1.1.1. Deleting workflows with dev profile

Before upgrading the Operator, you must delete workflows running with the **dev** profile and redeploy them after the upgrade is completed.

Procedure

1. Ensure you have a backup of all necessary Kubernetes resources, including **SonataFlow** custom resource definitions (CRDs), **ConfigMaps**, or any other related custom configurations.
2. Delete the workflow by executing the following command:
   -

```
$ oc delete -f <my-workflow.yaml> -n <target_namespace>
```

8.1.1.2. Deleting and migrating workflows with the preview profile

Before upgrading the Operator, you must delete workflows running with the **Preview** profile, and migrate any data that is persisted. When the upgrade is completed, you must redeploy the workflows.

Procedure

1. If you are using persistence, back up the workflow database and ensure the backup includes both database objects and table data.
2. Ensure you have a backup of all necessary Kubernetes resources, including **SonataFlow** CRDs, **ConfigMaps**, or any other related custom configurations.
3. Delete the workflow by executing the following command:

```
$ oc delete -f <my-workflow.yaml> -n <target_namespace>
```

4. If you are using persistence, you must execute the following database migration script:

```
ALTER TABLE flyway_schema_history
   RENAME CONSTRAINT flyway_schema_history_pk TO
kie_flyway_history_runtime_persistence_pk;

ALTER INDEX flyway_schema_history_s_idx
  RENAME TO kie_flyway_history_runtime_persistence_s_idx;

ALTER TABLE flyway_schema_history RENAME TO
kie_flyway_history_runtime_persistence;
```

8.1.1.3. Scaling down workflows with the gitops profile

Before upgrading the Operator, you must scale down workflows running with the **gitops** profile, and scale them up again after the upgrade is completed.

Procedure

1. Modify the **my-workflow.yaml** CRD and scale down each workflow to **zero** before upgrading as shown in the following example:

```
spec:
  podTemplate:
    replicas: 0
```

2. Apply the updated CRD by running the following command:

```
$ oc apply -f <my-workflow.yaml> -n <target_namespace>
```

3. (Optional) Scale the workflow to **0** by running the following command:

```
$ oc patch sonataflow <my-workflow> -n <target_namespace> --type=merge -p '{"spec":
{"podTemplate": {"replicas": 0}}}'
```

8.1.1.4. Backing up the Data Index database

You must back up the Data Index database before upgrading to prevent data loss.

Procedure

- Take a full backup of the Data Index database, ensuring:
  - The backup includes all database objects and not just table data.
  - The backup is stored in a secure location.

8.1.1.5. Backing up the Jobs Service database

You must back up the Jobs Service database before upgrading to maintain job scheduling data.

Procedure

- Take a full backup of the Jobs Service database, ensuring:
  - The backup includes all database objects and not just table data.
  - The backup is stored in a secure location.

8.1.2. Upgrading the OpenShift Serverless Logic Operator

To transition from OpenShift Serverless Logic Operator (OSL) version 1.34.0 to 1.35.0, you must upgrade the OSL using the Red Hat OpenShift Serverless web console. This upgrade ensures compatibility with newer features and proper functioning of your workflows.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to the **OpenShift Management Console** for Operator upgrades.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. In the web console, navigate to the **Operators → OperatorHub → Installed Operator** page.
2. Select the **openshift-serverless-logic** namespace from the **Installed Namespace**.
3. In the list of installed operators, find and click the OpenShift Serverless Logic Operator.
4. In the Operator details page, click on the **Subscription** tab. Click **Edit Subscription**.
5. In the **Upgrade status**, click the **Upgrade available** link.

6. Click the **Preview install plan** and **Approve** to start the update.

7. To monitor the upgrade process, run the following command:

```
$ oc get subscription logic-operator-rhel8 -n openshift-serverless-logic -o
jsonpath='{.status.installedCSV}'
```

Expected output

```
$ logic-operator-rhel8.v1.35.0
```

Verification

1. To verify the new Operator version is installed, run the following command:

```
$ oc get clusterserviceversion logic-operator-rhel8.v1.35.0 -n openshift-serverless-logic
```

Expected output

```
NAME                          DISPLAY                          VERSION   REPLACES
PHASE
logic-operator-rhel8.v1.35.0   OpenShift Serverless Logic Operator   1.35.0    logic-operator-
rhel8.v1.34.0   Succeeded
```

8.1.3. Finalizing the upgrade

After upgrading the OpenShift Serverless Logic Operator to version 1.35.0, you must finalize the upgrade process by restoring or scaling workflows and cleaning up old services. This ensures that your system runs cleanly on the new version and that all dependent components are configured correctly.

Follow the appropriate steps below based on the profile of your workflows and services.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have OpenShift Serverless Logic Operator installed on your cluster.
- You have access to a OpenShift Serverless Logic project with the appropriate roles and permissions to create applications and other workloads in OpenShift Container Platform.
- You have access to the **OpenShift Management Console** for Operator upgrades.
- You have installed the OpenShift CLI (**oc**).

8.1.3.1. Finalizing the Data Index upgrade

After the Operator upgrade, a new ReplicaSet is automatically created for Data Index 1.35.0. You must delete the old one manually.

Procedure

1. Verify the new ReplicaSet exists by listing all ReplicaSets by running the following command:

```
$ oc get replicasets -n <target_namespace> -o custom-
columns=Name:metadata.name,Image:spec.template.spec.containers[*].image
```

2. Identify the old Data Index ReplicaSet (with version 1.34.0) and delete it:

```
$ oc delete replicaset <old_replicaset_name> -n <target_namespace>
```

8.1.3.2. Finalizing the Job Service upgrade

You must manually clean up the Jobs Service components from the older version to trigger deployment of version 1.35.0 components.

Procedure

1. Delete the old Jobs Service deployment by running the following command:

```
$ oc delete deployment <jobs-service-deployment-name> -n <target_namespace>
```

   This will trigger automatic cleanup of the older Pods and ReplicaSets and initiate a fresh deployment using version 1.35.0.

8.1.3.3. Redeploying workflows with the dev profile

After the upgrade, you must redeploy workflows that use the **dev** profile and any associated Kubernetes resources.

Procedure

1. Ensure all required resources are restored, including **SonataFlow** CRDs, **ConfigMaps**, or any other related custom configurations.

2. Redeploy the workflow by running the following command:

```
$ oc apply -f <my-workflow.yaml> -n <target_namespace>
```

8.1.3.4. Restoring workflows with the preview profile

Workflows with the **preview** profile require an additional configuration step before being redeployed.

Procedure

1. If the workflow uses persistence, add the following property to the **ConfigMap** associated with the workflow:

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app: my-workflow
  name: my-workflow-props
data:
  application.properties: |
    kie.flyway.enabled=true
```

2. Ensure all required resources are recreated, including **SonataFlow** CRDs, **ConfigMaps**, or any other related custom configurations.

3. Redeploy the workflow by running the following command:

```
$ oc apply -f <my-workflow.yaml> -n <target_namespace>
```

8.1.3.5. Scaling up workflows with the gitops profile

Workflows with the **gitops** profile that were previously scaled down must be scaled back up to continue operation.

Procedure

1. Modify the **my-workflow.yaml** CRD and scale up each workflow to **one** before upgrading as shown in the following example:

```
spec:
  podTemplate:
    replicas: 1
```

2. Apply the updated CRD by running the following command:

```
$ oc apply -f <my-workflow.yaml> -n <target_namespace>
```

3. (Optional) Scale the workflow back to **1** by running the following command:

```
$ oc patch sonataflow <my-workflow> -n <target_namespace> --type=merge -p '{"spec":
{"podTemplate": {"replicas": 1}}}'
```

8.1.3.6. Verifying the upgrade

After restoring workflows and services, it is essential to verify that the upgrade was successful and that all components are functioning as expected.

Procedure

1. Check if all workflows and services are running by entering the following command:

```
$ oc get pods -n <target_namespace>
```

   Ensure that all pods related to workflows, Data Index, and Jobs Service are in a **Running** or **Completed** state.

2. Verify that the OpenShift Serverless Logic Operator is running correctly by entering the following command:

```
$ oc get clusterserviceversion logic-operator-rhel8.v1.35.0 -n openshift-serverless-logic
```

   Expected output

```
NAME                          DISPLAY                          VERSION   REPLACES
PHASE
logic-operator-rhel8.v1.35.0   OpenShift Serverless Logic Operator   1.35.0    logic-operator-
rhel8.v1.34.0   Succeeded
```

3. Check Operator logs for any errors by entering the following command:

```
$ oc logs -l control-plane=sonataflow-operator -n openshift-serverless-logic
```